Technical Notebook

CALPHAD Modeling of Copper Removal from Recycled Steel via Oxide Slag

Anthony DiMascio, Tim Kalmar, Aishwarya Muralidhar, Mohammed Oumer

MSE 4381 – Design & Professional Practice I
Advisor: Prof. Alan Luo | Post-Doc: Dr. Jianyue Zhang
Industry Partner: Honda R&D Americas (Dr. Jim Hu)

Spring 2026

Personal synthesis of methods, findings, and technical insights from the Honda CALPHAD copper removal project.

Living document, updated as new results come in.

CONTENTS

1 The Copper Problem in Steel Recycling **5**
1.1 Why This Matters . . . 5
1.2 Our Approach: CALPHAD Screening . . . 5
1.3 Project History . . . 5

2 Thermodynamic Framework **5**
2.1 How CALPHAD Works . . . 5
2.2 Key Quantity: GM (Gibbs Energy per Mole of Atoms) . . . 6
2.3 Mole Fraction Conventions . . . 6

3 Phase 1: Binary Screening (Ellingham Diagram) **6**
3.1 The Question . . . 6
3.2 Results . . . 6
3.3 Dr. Zhang's Pivot . . . 7

4 Phase 2: Ternary Reaction Modeling **7**
4.1 The Ternary Reaction . . . 7
4.2 Results: All 16 Reactions Are Favorable . . . 8
4.3 Dr. Zhang and FeO: Binary vs. Ternary . . . 8

5 Activity Correction for Dilute Copper **9**
5.1 The Problem with Pure ΔG . . . 9
5.2 The Penalty Term . . . 9
5.3 Impact on Candidate Ranking . . . 9
5.4 γ_{Cu} Sensitivity . . . 10

6 Phase Persistence and Slag Chemistry **11**
6.1 No Ternary Phase Survives at 1800 K . . . 11
6.2 The Ionic Liquid Model in TCOX14 . . . 12
6.3 Slag Basicity: The Oxide "Ionic Atmosphere" . . . 12
6.3.1 The Aqueous Analogy . . . 12
6.3.2 The Slag Equivalent . . . 12
6.3.3 Modeling Approaches Compared . . . 12
6.3.4 Thermodynamics vs. Kinetics of Mixing . . . 13
6.4 Slag Basicity Results . . . 13

7 Cu Activity and the Gibbs Phase Rule **13**
7.1 The "Flat Activity" Puzzle . . . 13
7.2 Explanation: Gibbs Phase Rule . . . 14
7.3 The Missing Metal Phase . . . 14
7.4 The $a_{\mathrm{Cu}} \approx 0.0006$ Value . . . 15

8 Ternary Phase Maps at 1800 K **15**

9 TC-Python Database Landscape **18**
9.1 Thermodynamic Databases . . . 18
9.2 Mobility Databases (for DICTRA) . . . 19
9.3 API Lessons . . . 19

10 DICTRA and Diffusion Kinetics **19**
10.1 What DICTRA Does . . . 19
10.2 Why It Matters for Cu Removal . . . 20
10.3 TC-Python Diffusion API . . . 20
10.4 DICTRA Capability Test Results (March 14, 2026) . . . 20
10.4.1 Phase 1–2: Database Compatibility & API Discovery . . . 20
10.4.2 Phase 3–6: Initial Failures and Script Bug Fixes . . . 21
10.4.3 Re-Run Results (March 14 PM: Fixed Script) . . . 21
10.4.4 Run 3 Results: Phases 4–6 (March 14, 6:42 PM) . . . 21
10.4.5 Implications . . . 22

11 $CuFe_2O_4$: Decomposing the Driving Force **22**
11.1 Method . . . 23
11.2 Result . . . 23

12 V_2O_5: Two Products, Two Stories **23**

13 Toward Experimental Validation **23**
13.1 Recommended Candidates . . . 23
13.2 Mass Balance Estimates . . . 24
13.3 Cu Removal Rate Prediction (DICTRA) . . . 24
13.4 Experiment Plan . . . 26

14 Technical Pitfalls and Lessons Learned **26**

15 Glossary **27**

A Chronological Lab Entries **28**
A.1 January 23–26, 2026: Project Kickoff . . . 28
A.2 January 31 – February 4, 2026: First Calculations . . . 28
A.3 February 12, 2026: Dr. Zhang Meeting: Direction Shift . . . 28
A.4 March 12, 2026: Screening Pipeline Complete . . . 29
A.5 March 13, 2026: Ternary Breakthrough . . . 29
A.6 March 14, 2026: DICTRA Kinetics Testing . . . 29
A.7 March 14, 2026: Unified Workbook . . . 30
A.8 March 14, 2026: Experiment Design Tools . . . 31
A.9 March 14, 2026: DICTRA Cu Removal Rate — 20/20 Success . . . 31

B Script Index **32**
B.1 VM Scripts (`simulations/tcpython/`) . . . 32
B.2 Local Post-Processing Scripts (`screening/`) . . . 32
B.3 Marimo Notebooks (`simulations/notebooks/`) . . . 33

C Data Files **34**

D Selected Scripts **35**
D.1 extract_oxide_gibbs.py . . . 35
D.2 extract_ternary_reactions.py . . . 41
D.3 cufe2o4_alternative_reaction.py . . . 52
D.4 cu_activity_vs_oxide.py . . . 58
D.5 slag_composition_effects.py . . . 62

D.6 compute_ternary_dG.py 66
D.7 compute_activity_corrected_dG.py 69

1 THE COPPER PROBLEM IN STEEL RECYCLING

1.1 Why This Matters

Recycled steel picks up copper from wiring, motors, and electronics in automotive scrap. Typical Cu levels run 0.25–0.3 wt% and keep rising as vehicles get more electronic. The problem starts above about 0.1 wt%: Cu segregates to grain boundaries during hot working and causes **"hot shortness"**, surface cracking that ruins the steel for body panels [1].

Nobody has a good industrial-scale fix. The standard workaround is diluting with virgin iron or DRI, which is expensive and defeats the purpose of recycling [1]. Honda uses a lot of sheet steel, so they have a direct stake in solving this.

1.2 Our Approach: CALPHAD Screening

CALPHAD (Calculation of Phase Diagrams) uses thermodynamic databases to predict which phases are stable at a given temperature and composition [2]. Our strategy:

1. Identify oxide materials that might *capture* Cu from molten steel
2. Compute the Gibbs free energy of Cu-capture reactions ($\Delta G < 0$ = favorable)
3. Rank candidates by thermodynamic driving force
4. Validate top candidates experimentally in Fontana Lab

The key tool is TC-Python [3, 15], the Python API for Thermo-Calc, using the TCOX14 oxide database [4] and TCFE13 steel database.

1.3 Project History

This is **Year 3** of Honda's decopperization project at OSU:

- **Year 1** (Ken Kushner): Initial scoping
- **Year 2** (Andrew's group): Experimental tests with Al_2O_3. Some Cu reduction was observed, but Na_2S salt flux was more effective (95% removal per Dr. Zhang, Feb 12 meeting). Na_2S produces toxic H_2S gas, motivating the oxide approach.
- **Year 3** (our team): Systematic CALPHAD screening of oxide candidates

2 THERMODYNAMIC FRAMEWORK

2.1 How CALPHAD Works

CALPHAD models the Gibbs energy of every phase as a function of temperature and composition [2, 7]. For a phase ϕ:

$$G_m^\phi(T, \mathbf{x}) = \sum_i x_i \, {}^0G_i^\phi(T) + RT \sum_i x_i \ln x_i + G^{\text{xs}}(T, \mathbf{x}) \tag{1}$$

Three terms: (1) reference energy from pure components, (2) ideal mixing entropy (the $RT \ln x$ term you see in every thermo textbook [6]), and (3) excess Gibbs energy for non-ideal interactions. The third term is where all the real work goes; it gets fitted to experimental data and DFT calculations [2].

At equilibrium, the system minimizes total G by distributing components across phases. TC-Python [3] does this numerically for systems with dozens of phases.

2.2 Key Quantity: GM (Gibbs Energy per Mole of Atoms)

TC-Python returns `GM`, the Gibbs energy per mole of *atoms*, not per mole of formula units [3]. This tripped us up early:

GM Normalization

For a compound with N atoms per formula unit (e.g., $CuFe_2O_4$ has $N = 7$):

$$G_{\text{per formula}} = N \times \texttt{GM}$$

Getting this wrong gives energies off by factors of 5–7×. We caught this bug early.

Similarly, the O_2 reference requires:

$$G_{O_2} = 2 \times \texttt{GM}(\text{pure O at same } T)$$

because `GM` is per O atom.

2.3 Mole Fraction Conventions

In TC-Python, you set $N - 1$ mole fraction conditions for an N-component system. The last element fills the remainder. For a ternary Cu-Fe-O system modeling $CuFe_2O_4$ ($x_{Cu} = 1/7$, $x_{Fe} = 2/7$, $x_O = 4/7$):

```
calc.set_condition("X(CU)", 1.0/7.0)
calc.set_condition("X(FE)", 2.0/7.0)
# O fills remainder: X(O) = 4/7
```

3 PHASE 1: BINARY SCREENING (ELLINGHAM DIAGRAM)

3.1 The Question

Can metallic Cu directly reduce a ceramic oxide? For example:

$$Cu + MgO \rightarrow Cu_2O + Mg \quad \text{(does this happen?)}$$

You answer this by comparing standard Gibbs energies of oxide formation on an Ellingham diagram [6, 13]. If Cu_2O is *less stable* than the target oxide, Cu cannot reduce it.

3.2 Results

Cu_2O is the **least stable oxide** in our study, with a formation energy approximately 800 kJ mol^{-1} O_2 less negative than Al_2O_3, MgO, or SiO_2. This means:

Binary Screening Conclusion

Cu **cannot reduce any useful ceramic oxide**. All $\Delta G_{rxn} > 0$ for the binary exchange reaction. This rules out the simplest Cu removal mechanism.

We extracted formation energies for 18 oxides (11 from TCOX14 [4], 2 from SSUB3 [5], 7 "moonshot" literature values) over 500–2000 K in 50 K steps. Every one confirms: Cu cannot reduce any of them at steelmaking temperatures.

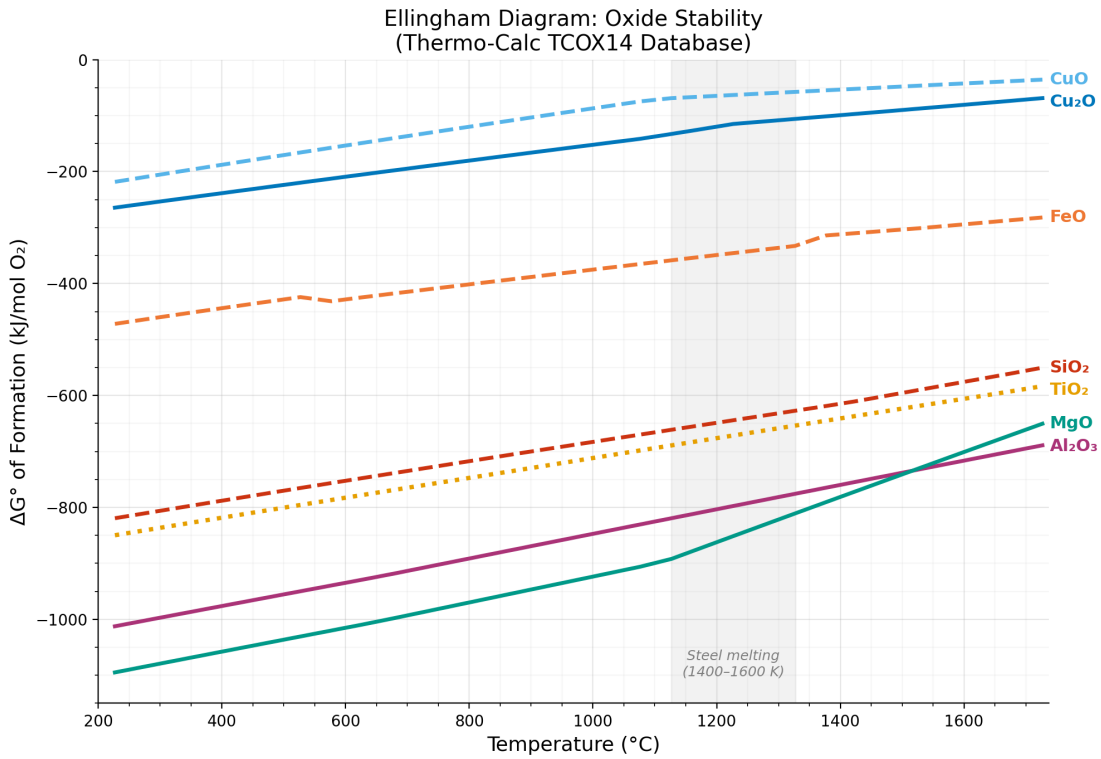

Figure 1: Ellingham diagram of oxide formation energies from TCOX14. Cu_2O (red) sits far above all candidate oxides, confirming Cu cannot reduce any of them. Script: `simulations/notebooks/ellingham_diagram.py`.

3.3 Dr. Zhang's Pivot

At our Feb 12 meeting, Dr. Zhang explained that the binary result was *expected*. The real mechanism is not oxide reduction but **ternary compound formation**:

> "We want to use the oxide to capture the copper. Last time, [Al_2O_3] worked because of the spinel reaction."

The reaction is not Cu + Al_2O_3 → products, but rather:

$$\mathrm{Cu} + \mathrm{Al_2O_3} + \tfrac{1}{2}\mathrm{O_2} \rightarrow \mathrm{CuAl_2O_4} \quad \text{(spinel)}$$

That is a completely different thermodynamic question.

4 PHASE 2: TERNARY REACTION MODELING

4.1 The Ternary Reaction

For each candidate oxide MO_x, we compute:

$$n_{\mathrm{Cu}} \cdot \mathrm{Cu} + a \cdot \mathrm{MO}_x + \tfrac{b}{2}\,\mathrm{O_2} \rightarrow \mathrm{Cu}_n\mathrm{M}_c\mathrm{O}_d \tag{2}$$

where the stoichiometry comes from the known ternary compound. The standard free energy of reaction [6]:

$$\Delta G_{\mathrm{rxn}} = G_{\mathrm{product}} - \left[n_{\mathrm{Cu}} \cdot G_{\mathrm{Cu}} + a \cdot G_{\mathrm{MO}_x} + \tfrac{b}{2} \cdot G_{\mathrm{O_2}}\right] \tag{3}$$

Each G term is computed by TC-Python as a single-phase equilibrium at the compound's stoichiometric composition. `GM` × (atoms per formula unit) gives per-formula energies.

4.2 Results: All 16 Reactions Are Favorable

Every ternary reaction we modeled has $\Delta G < 0$ at 1527 °C (1800 K):

Table 1: Top Ternary Reaction Candidates, Ranked by ΔG at 1800 K

Rank	Product	ΔG (kJ)	Parent Oxide	n_{Cu}
1	$CuFe_2O_4$	−111.9	FeO	1
2	$Cu_3V_2O_8$	−109.2	V_2O_5	3
3	$CuMn_2O_4$	−64.4	MnO	1
4	Cu_2SiO_4	−52.2	SiO_2	2
5	CuB_2O_4	−47.7	B_2O_3	1
6	CuV_2O_6	−45.0	V_2O_5	1
7	$CuAl_2O_4$	−32.3	Al_2O_3	1

Note that the pure-ΔG ranking is misleading under real conditions. See Section 5 for the activity correction.

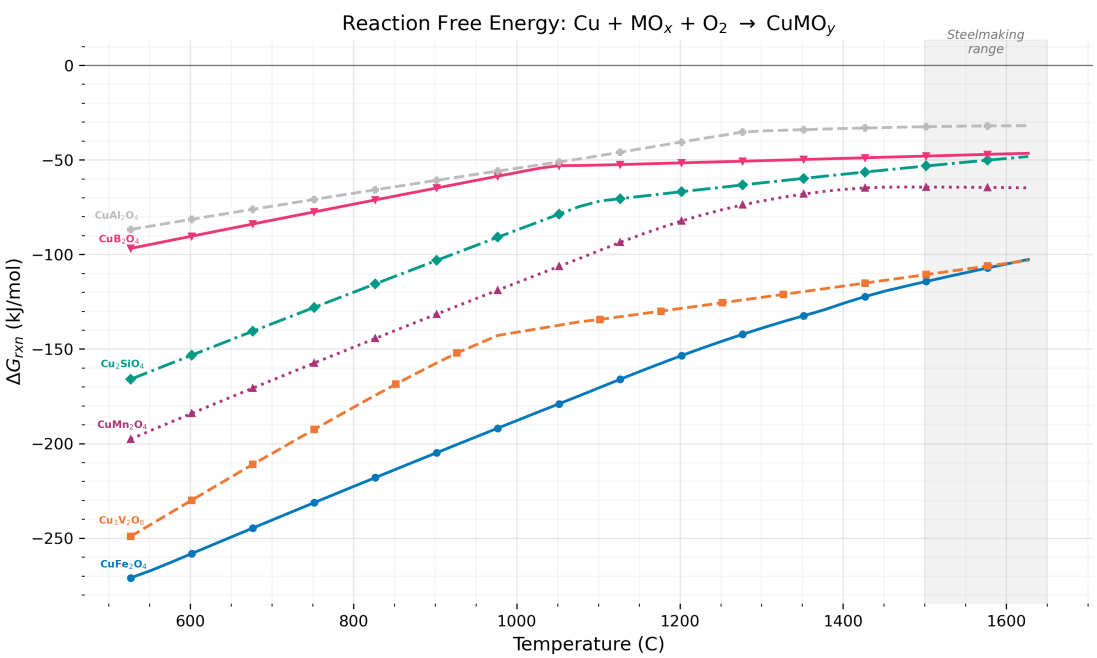

Figure 2: ΔG vs. temperature for the top 6 ternary reaction candidates. All reactions become less favorable at higher temperatures, as expected from entropy considerations. Kinks indicate phase transitions (e.g., spinel melting). Script: `screening/plot_dG_vs_T.py`.

4.3 Dr. Zhang and FeO: Binary vs. Ternary

When asked about FeO, Dr. Zhang said:

> "Iron oxide should not work. I'm pretty sure it doesn't, but I'm not certain."

He was talking about the **binary** reaction (Cu reducing FeO), which is correct; that does not work. Our calculations show a different reaction entirely: $Cu + 2FeO + O_2 \rightarrow CuFe_2O_4$ (spinel formation). This **ternary** reaction has the strongest driving force of any candidate (−112 kJ). Dr. Zhang himself asked us to check it.

5 ACTIVITY CORRECTION FOR DILUTE COPPER

5.1 The Problem with Pure ΔG

The ΔG values in Table 1 assume **pure Cu** ($a_{\mathrm{Cu}} = 1$). In recycled steel, Cu is dilute: $x_{\mathrm{Cu}} \approx 0.003$ (0.3 wt%). Dilute Cu has lower chemical potential [6], so it is harder to capture.

5.2 The Penalty Term

For each Cu atom consumed, there is an activity correction:

$$\Delta G_{\mathrm{penalty}} = -RT \ln a_{\mathrm{Cu}} = -RT \ln(\gamma_{\mathrm{Cu}} \cdot x_{\mathrm{Cu}}) \tag{4}$$

Using $\gamma_{\mathrm{Cu}} = 8.5$ (Raoultian activity coefficient for Cu in liquid Fe at 1800 K, from Hino & Ito [9]; Sigworth & Elliott [10] give ∼10) and $x_{\mathrm{Cu}} = 0.003$:

$$a_{\mathrm{Cu}} = 8.5 \times 0.003 = 0.0255$$

$$\Delta G_{\text{penalty per Cu atom}} = -RT \ln(0.0255) = +54.9\,\mathrm{kJ\,mol^{-1}} \quad \text{at } 1800\,\mathrm{K}$$

The total correction scales with the number of Cu atoms in the product:

$$\Delta G_{\mathrm{eff}} = \Delta G_{\mathrm{pure}} + n_{\mathrm{Cu}} \times \Delta G_{\mathrm{penalty}} \tag{5}$$

5.3 Impact on Candidate Ranking

Activity Correction Reshuffles Everything

Products with multiple Cu atoms per formula unit get penalized heavily:

- $CuFe_2O_4$ ($n_{\mathrm{Cu}} = 1$): $-111.9 + 54.9 = \mathbf{-57.0}$ kJ (still strongly favorable)
- $CuMn_2O_4$ ($n_{\mathrm{Cu}} = 1$): $-64.4 + 54.9 = -9.5$ kJ (marginal)
- $Cu_3V_2O_8$ ($n_{\mathrm{Cu}} = 3$): $-109.2 + 3 \times 54.9 = +55.6$ kJ (**unfavorable!**)

The pure-ΔG #2 candidate ($Cu_3V_2O_8$) becomes unfavorable once dilution is accounted for. But V_2O_5 can also form CuV_2O_6 ($n_{\mathrm{Cu}} = 1$), which gives $-45.0 + 54.9 = +9.9$ kJ. Uncertain, but not clearly dead. At higher Cu concentrations (∼1 wt%), the activity penalty shrinks and both V_2O_5 products could become favorable.

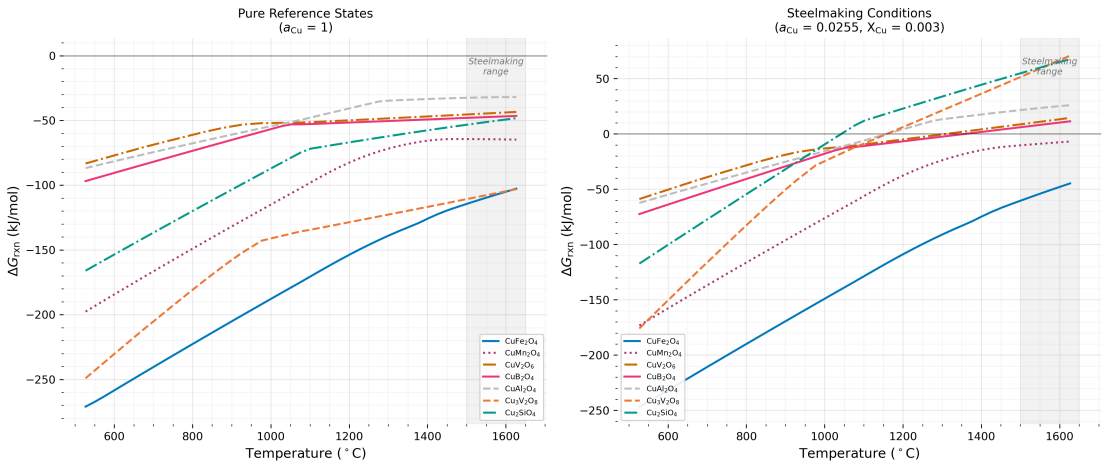

Figure 3: Activity-corrected ΔG_{eff} vs. pure ΔG for all candidates at 1800 K. Products with $n_{\text{Cu}} \geq 2$ ($Cu_3V_2O_8$, Cu_2SiO_4) shift dramatically rightward. Only products to the left of the dashed line ($\Delta G_{\text{eff}} < 0$) are viable under dilute conditions. Script: `screening/compute_activity_corrected_dG.py`.

5.4 γ_{Cu} Sensitivity

The literature range for γ_{Cu} is roughly 5–13 [9, 10]. This shifts the penalty by about ±10 kJ:

- $CuFe_2O_4$ is **favorable** across all γ_{Cu} values (always < -40 kJ)
- $CuMn_2O_4$ crosses zero at $\gamma_{\text{Cu}} \approx 5$ (marginal)
- CuB_2O_4 and CuV_2O_6 hover near zero (uncertain)
- Everything with $n_{\text{Cu}} \geq 2$ is unfavorable under steelmaking dilution

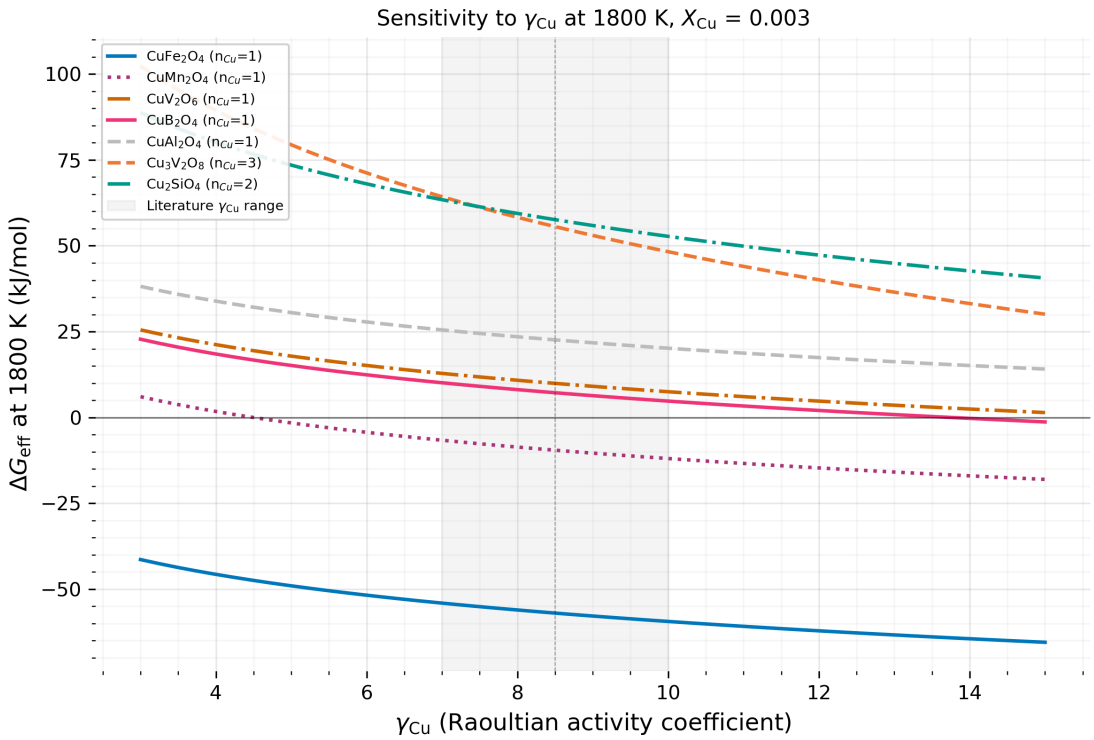

Figure 4: Sensitivity of ΔG_{eff} to the Cu activity coefficient γ_{Cu} (range 5–13). $CuFe_2O_4$ stays favorable across the entire range. $CuMn_2O_4$ is marginal. Script: `screening/compute_activity_corrected_dG.py`.

6 PHASE PERSISTENCE AND SLAG CHEMISTRY

6.1 No Ternary Phase Survives at 1800 K

A critical finding: while the ternary reactions are thermodynamically favorable, **no crystalline ternary compound exists at steelmaking temperature**. All named phases (spinels, delafossites, vanadates) melt below 1800 K:

- $CuFe_2O_4$ spinel: stable to 1700 K (1427 °C), closest to steelmaking
- $CuMn_2O_4$ spinel: stable to 1700 K
- $CuAl_2O_4$ spinel: stable to 1550 K (1277 °C)
- $CuCr_2O_4$ spinel: stable to 1450 K
- All other products: always liquid (IONIC_LIQ in TC-Python)

Cu Capture = Slag Dissolution

At steelmaking temperatures, copper is not captured into a crystalline compound [8]. Instead, it **dissolves into the oxide slag** (ionic liquid). The negative ΔG values still represent a real thermodynamic driving force; they describe the energy released when Cu enters the slag phase. The mechanism is dissolution, not precipitation.
This may actually be advantageous for experiments: a liquid slag has much better contact with molten steel than a solid particle would.

6.2 The Ionic Liquid Model in TCOX14

TCOX14 [4] models oxide melts using the **IONIC_LIQ** phase, a two-sublattice model developed by Hillert et al. [7]: one sublattice holds cations (Cu^{1+}, Cu^{2+}, Fe^{2+}, Fe^{3+}, Al^{3+}, etc.) and the other holds anions (O^{2-}, vacancies, neutral species like SiO_2). The excess Gibbs energy captures pairwise interactions between ions on each sublattice. This is how TC-Python knows that Cu prefers certain anion arrangements (like a spinel-like local environment) even when the bulk phase is liquid.

6.3 Slag Basicity: The Oxide "Ionic Atmosphere"

If you have done any aqueous solution chemistry, this will feel familiar.

6.3.1 The Aqueous Analogy

In aqueous solutions (e.g., OLI simulations with $CaCl_2$, NaCl, KCl), the ions around a dissolved species modify its effective activity [6]. Adding more salt increases ionic strength, which changes what dissolves and what precipitates. Debye-Hückel theory describes this. Order of addition can matter too, because local supersaturation at the mixing point can nucleate different solids before the system reaches equilibrium.

6.3.2 The Slag Equivalent

In molten oxide slags, the analogous variable is **slag basicity**: the ratio of basic oxides (CaO, MnO, FeO, Na_2O) to acidic oxides (SiO_2, Al_2O_3) [11]. Basicity modifies the activity coefficients of everything in the melt, including Cu. Adding CaO (a strong base) to a silicate slag can shift $\gamma_{CuO_{0.5}}$ significantly, making Cu easier or harder to dissolve. The surrounding Ca^{2+}, Fe^{2+}, O^{2-} ions modify the effective chemical potential of Cu in the slag, just like ionic strength modifies solute activity in water. Dr. Zhang's suggestion of adding Na_2S flux is exactly this idea: change the "ionic atmosphere" of the slag to improve Cu capture.

6.3.3 Modeling Approaches Compared

Table 2: Activity Coefficient Models: Aqueous vs. Oxide Melts

Feature	Aqueous (OLI)	Oxide Slag (TCOX14)
Model	MSE (Mixed Solvent Electrolyte)	Ionic Two-Sublattice
Core concept	Excess G from pairwise ion interactions	Excess G from pairwise ion interactions
Reference state	Infinite dilution in water	Pure oxide liquid
Temperature range	25–300 °C	1200–1800 °C
Key variable	Ionic strength	Slag basicity
Software	OLI Studio	Thermo-Calc (TCOX14)
Competing approach	Pitzer model	Modified Quasichemical Model (FactSage)

The physics is the same in both cases: excess Gibbs energy from pairwise ion interactions determines activity coefficients. The only differences are the solvent (water vs. oxide melt) and the temperature range.

FactSage uses the Modified Quasichemical Model (MQM) instead of the two-sublattice approach. Some slag chemists prefer MQM because it handles short-range ordering in silicate melts better [11]. In practice, both are fitted to the same experimental data and give similar results for most systems.

6.3.4 Thermodynamics vs. Kinetics of Mixing

Thermodynamically, the final slag composition determines equilibrium regardless of mixing order. Gibbs energy reaches the same minimum either way [6].

Kinetically, order matters. If you dump MnO into a silicate slag, local supersaturation at the addition point can nucleate $MnSiO_3$ crystals before the system mixes. Those metastable precipitates might trap or release Cu differently than the equilibrium slag would. Modeling these transient gradients is a **DICTRA problem** [3]: you need diffusion simulation, not just equilibrium thermo.

6.4 Slag Basicity Results

Script 5 (`slag_composition_effects.py`) tested two quaternary systems at 1800 K. In Cu-Mn-Si-O, increasing the $MnO:SiO_2$ ratio *decreases* a_{Cu} by 17%, meaning a more basic slag captures Cu more effectively. That is a tunable parameter for experiments. In Cu-Al-Si-O, varying $Al_2O_3:SiO_2$ has almost no effect on a_{Cu}; the alumina is essentially inert as far as Cu activity goes.

A real steelmaking slag is CaO-SiO_2-Al_2O_3-FeO-MnO-MgO. Future work should explore quinary and higher-order systems (e.g., Cu-Fe-Mn-Ca-Si-O) to map how basicity affects Cu capture in realistic compositions.

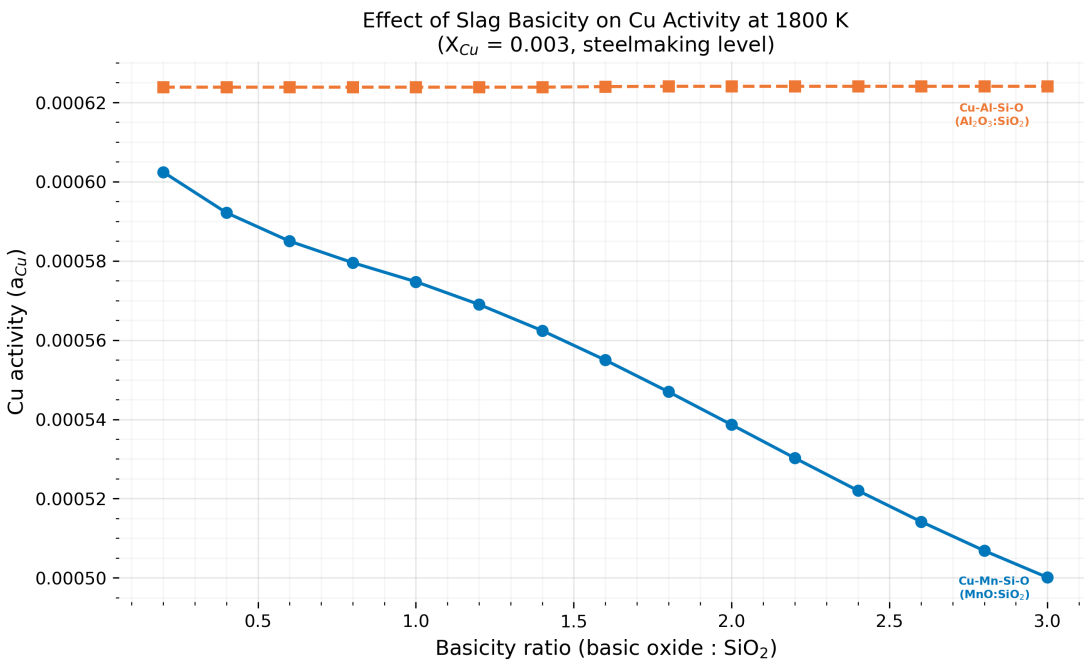

Figure 5: Cu activity vs. slag basicity ratio for Cu-Mn-Si-O and Cu-Al-Si-O at 1800 K. Increasing $MnO:SiO_2$ (more basic) reduces a_{Cu} by 17%, while $Al_2O_3:SiO_2$ has negligible effect. Script: `screening/plot_slag_effects.py`.

7 CU ACTIVITY AND THE GIBBS PHASE RULE

7.1 The "Flat Activity" Puzzle

Script 2 (`cu_activity_vs_oxide.py`) swept Cu mole fraction from 0.90 to 0.05 in four ternary systems at 1800 K, recording a_{Cu} via `AC(CU)` in TC-Python. The result was puzzling: a_{Cu} was essentially **constant** ($\sim 6 \times 10^{-4}$) across the entire composition range for Cu-Al-O, Cu-Mn-O, and Cu-Fe-O.

7.2 Explanation: Gibbs Phase Rule

This is not a code bug. It follows directly from the **Gibbs Phase Rule** [6, 14]:

$$F = C - P + 2$$

At fixed T and P (F reduced by 2), for a 3-component system ($C = 3$) in a 2-phase field ($P = 2$):

$$F = 3 - 2 + 2 - 2 = 1$$

But we have already fixed T and P, so $F = 1$. In a 2-phase field, the compositions of both phases are **fixed**; only the relative *amounts* change as you move along a tie line (lever rule). Since phase compositions are fixed, activity is invariant.

Why a_{Cu} Is Flat

In TCOX14 at 1800 K, most Cu-M-O systems sit in a 2-phase field (e.g., CORUNDUM + IONIC_LIQ for Cu-Al-O). Sweeping overall composition just moves along the tie line between these two phases without changing either phase's composition. The Cu activity, which depends on the phase compositions (not the overall composition), stays constant.

7.3 The Missing Metal Phase

TCOX14 has **no separate metallic Cu phase** (no FCC_A1 or metallic LIQUID). All Cu dissolves into the IONIC_LIQ (two-sublattice ionic liquid model). This means TCOX14 cannot model the steel/slag interface; it only sees the slag side.

To get a_{Cu} in metallic Cu coexisting with oxide slag, you would need a database containing **both** metallic phases (from TCFE) and oxide phases (from TCOX). Whether TCFE13 + TCOX14 can be combined is an open question tied to the DICTRA testing (Section 10).

7.4 The $a_{\mathbf{Cu}} \approx 0.0006$ Value

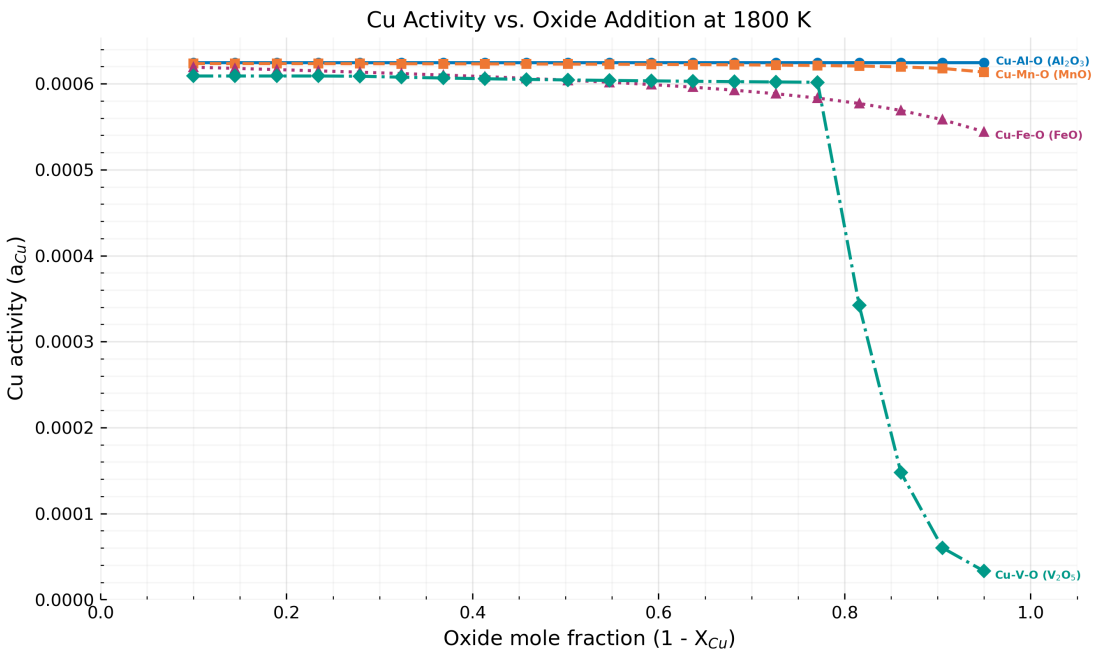

Figure 6: Cu activity (a_{Cu}) vs. mole fraction of Cu in four ternary systems at 1800 K. Activity is flat across the entire sweep for Cu-Al-O, Cu-Mn-O, and Cu-Fe-O (Gibbs Phase Rule: 2-phase field constrains compositions). Cu-V-O shows a drop to 3×10^{-5} at high oxide fraction. Script: `screening/plot_cu_activity.py`.

The flat value of $a_{\mathrm{Cu}} \approx 6 \times 10^{-4}$ *is* physically meaningful. It represents the equilibrium Cu activity set by the Cu-O redox couple in the ionic liquid at 1800 K. In the Cu-V-O system, a_{Cu} drops further to $\sim 3 \times 10^{-5}$ at high oxide fractions, indicating that V_2O_5 captures Cu most aggressively among the systems tested.

8 TERNARY PHASE MAPS AT 1800 K

Script 5 (`ternary_phase_map_1800K.py`) mapped phase stability across 20×20 composition grids in three Cu-M-O systems at 1800 K. These maps reveal which phases dominate at steelmaking temperature and confirm that most of the composition space is molten.

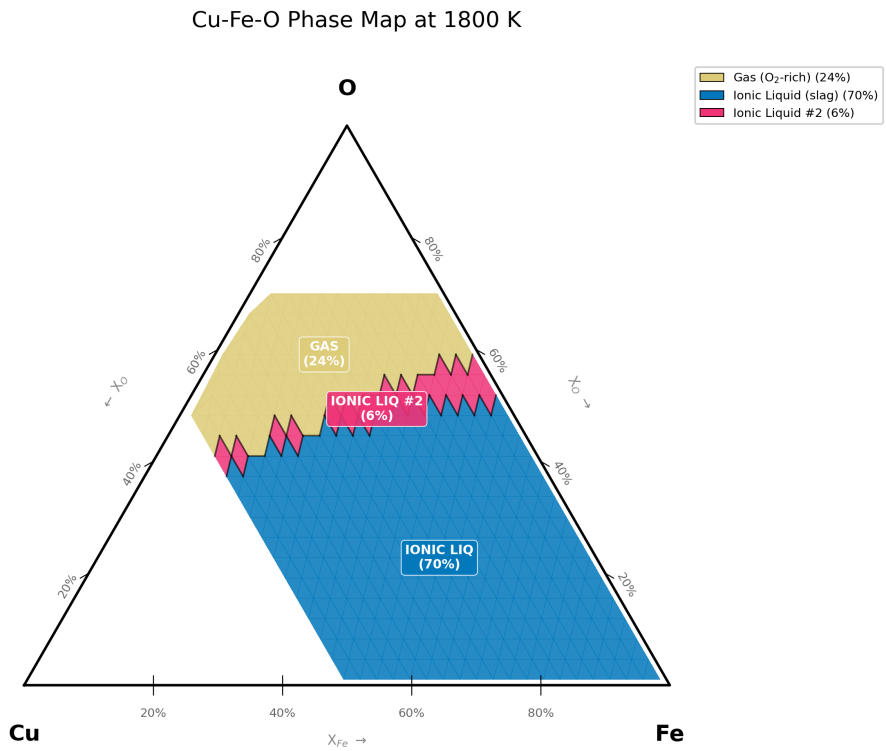

Figure 7: Phase map for Cu-Fe-O at 1800 K. IONIC_LIQ dominates (∼70% of points), confirming the system is fully liquid at steelmaking temperature. Script: `screening/plot_ternary_phase.py`.

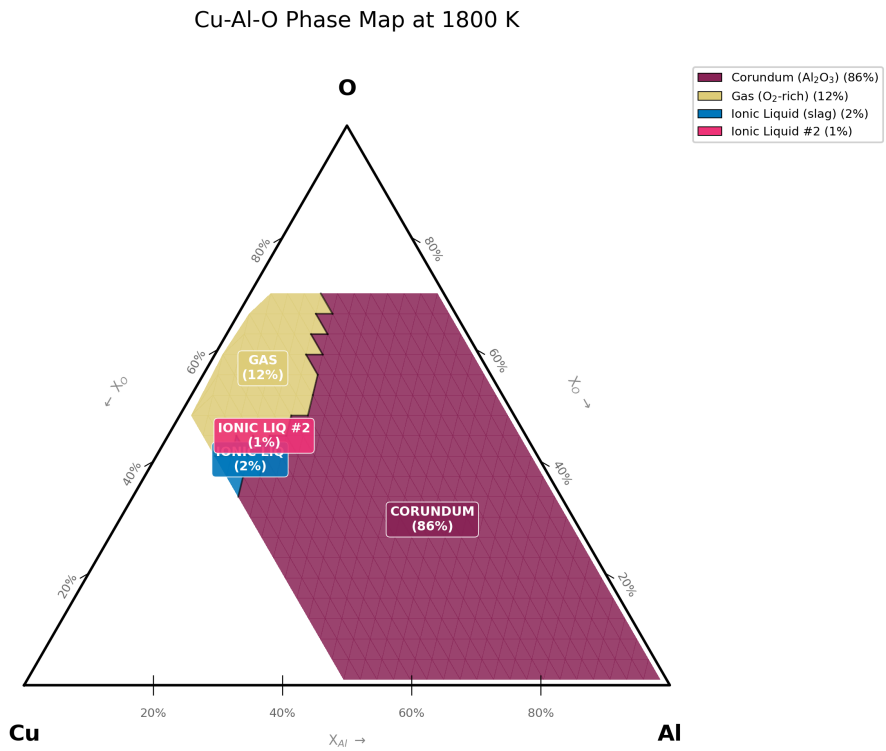

Figure 8: Phase map for Cu-Al-O at 1800 K. Large CORUNDUM (Al_2O_3) region persists (∼86% of points), reflecting Al_2O_3's high melting point (2072 °C). Cu-rich corner is ionic liquid.

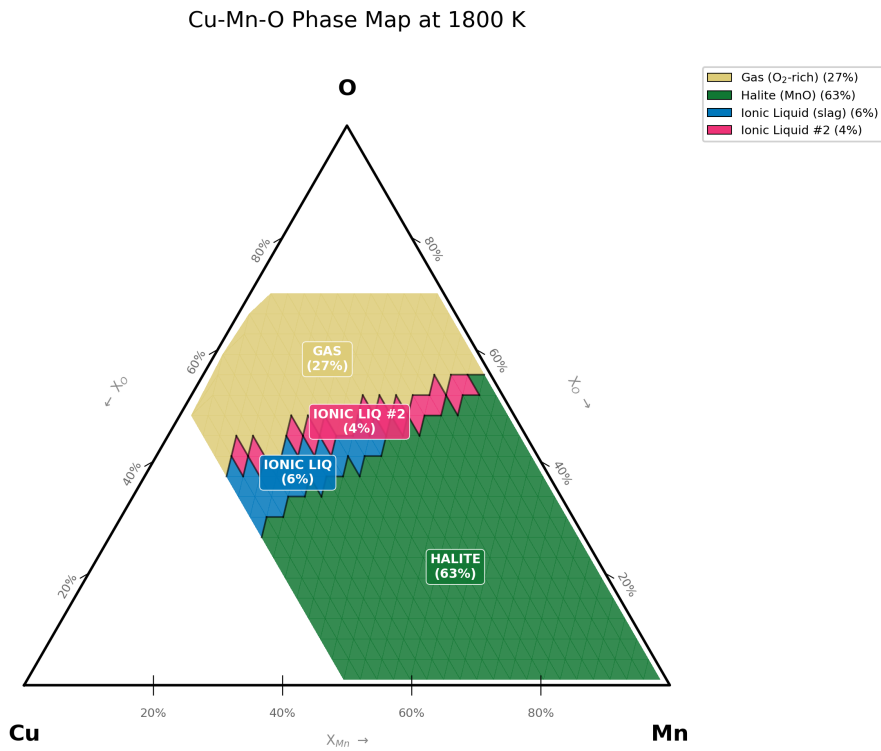

Figure 9: Phase map for Cu-Mn-O at 1800 K. HALITE (MnO rock-salt) phase dominates (∼63%), with ionic liquid in the Cu-rich and O-rich corners.

9 TC-PYTHON DATABASE LANDSCAPE

9.1 Thermodynamic Databases

Table 3: Thermodynamic Databases on the OSU VM

Database	Status	Notes
TCOX14	Available	Oxide systems. Our primary DB. Contains IONIC_LIQ, SPINEL, CORUNDUM, etc. No metallic phases.
TCFE13	Available	Steel systems. Contains FCC_A1, BCC_A2, LIQUID, CEMENTITE, etc. Limited oxide coverage.
TCFE12, TCFE11	Available	Older steel databases. Same phase types as TCFE13.
SSUB3	Available	Pure substance database. Fallback for elements not in TCOX14 (used for PbO, CeO_2).

9.2 Mobility Databases (for DICTRA)

Table 4: Mobility Databases Tested on the OSU VM (March 14, 2026)

Database	Status	Notes
MOBFE8	Found	Loads with TCFE13, TCFE12, TCFE11, **and TCOX14**. Primary steel mobility DB.
MOBFE7, 6, 5	Found	Older versions. All load with all TCFE and TCOX14.
MOBFE9	Not found	Not installed on this VM.
MOBOX1, MOBOX2	Not found	Oxide mobility. Files may exist on disk but TC engine cannot load them (Error 1234).
MOBCU3, MOBCU2	Found	Copper-specific mobility. SER reference state warning for O-containing systems. MOBCU2 fails with Cu-Fe-O on TCFE13.
MOBNI5, MOBNI4	Found	Nickel mobility. Creates FCC_L12 and BCC_B2 ordered composition sets.
MOBAL3	Found	Aluminum mobility. "O INPUT IGNORED" warning on O-containing systems but still loads.
MOBAL2	Partial	Loads without O. Cu-Fe-O and Cu-Fe-Mn-O crash with calculation error.

9.3 API Lessons

1. **Empty element list is a false positive**: `select_database_and_elements(db, [])` with an empty element list just checks if the database *file* exists, not whether it can actually load. MOBOX1/MOBOX2 files exist on disk but the TC engine throws Error 1234 when trying to parse them. Always test with real elements.

2. **Kinetic System API differs from thermodynamic**: The `System` object returned by `select_thermodynamic_and_kinetic_databases_with_elements()` does **not** have `get_phase_names()`. Instead, use `get_phases_in_system()` and `get_elements_in_system()`.

3. **VM workflow**: Always download ZIP from GitHub, extract, and run. Never `git pull` on the VM (git may not be installed, and Citrix clipboard corrupts file transfers).

10 DICTRA AND DIFFUSION KINETICS

10.1 What DICTRA Does

DICTRA (DIffusion Controlled TRAnsformations) solves the multicomponent diffusion equation:

$$\frac{\partial c_i}{\partial t} = \nabla \cdot \left(\sum_j D_{ij} \nabla c_j \right) \tag{6}$$

where D_{ij} is the interdiffusion coefficient matrix, computed from mobility data at each time step using local thermodynamic equilibrium (the "DICTRA assumption").

In TC-Python, DICTRA is accessed via:

```
system = session.select_thermodynamic_and_kinetic_databases_with_elements(
    "TCFE13", "MOBFE8", ["CU", "FE"]).get_system()
diff_calc = system.with_isothermal_diffusion_calculation()
```

10.2 Why It Matters for Cu Removal

Thermodynamics tells us *whether* Cu capture is favorable. DICTRA [3] tells us *how fast*:

- How quickly does Cu diffuse from bulk steel to the steel/slag interface?
- How long must the slag contact the melt to achieve meaningful Cu removal?
- Does the geometry matter (flat interface vs. spherical oxide particles)?

If Cu diffusion in liquid Fe is fast ($D \sim 10^{-9}$ m^2/s at 1800 K [9]), then a 10-minute furnace hold might remove Cu from a 1-cm-thick steel layer. If it is slow, we might need stirring, smaller particles, or longer treatment times.

10.3 TC-Python Diffusion API

The API provides:

- **Region**: A 1D spatial domain with width, grid, and initial composition profile
- **CalculatedGrid**: Linear, geometric, or double-geometric point distributions
- **ElementProfile**: Constant, linear, or step initial composition within a region
- **BoundaryCondition**: Closed system, fixed composition, or activity-based flux
- **Geometry**: Planar, cylindrical, or spherical
- **Solver**: Automatic, Classic, or Homogenization

The activity-based boundary condition is particularly relevant: it allows us to impose $a_{\mathrm{Cu}} = 0.0006$ (the equilibrium value from TCOX14) at the steel surface, simulating the chemical potential drop that an oxide slag imposes. This avoids needing to model the slag phase explicitly.

10.4 DICTRA Capability Test Results (March 14, 2026)

A 6-phase DICTRA test script ran on the OSU VM (∼1 hour runtime). Results split into two categories: API discovery (Phases 1–2, succeeded) and actual diffusion calculations (Phases 3–6, failed due to fixable script bugs).

10.4.1 Phase 1–2: Database Compatibility & API Discovery

Of 312 thermo × mobility combinations tested, **232 loaded successfully** and **40 unique pairs** support isothermal diffusion calculations. Key findings:

- MOBFE8/7/6/5 all load with TCFE13/12/11 and TCOX14
- MOBFE9, MOBOX1, MOBOX2: **not found** on this VM
- MOBCU3 loads for all element sets (copper-specific mobility)
- The kinetic System object has 20 public methods; the correct phase enumeration method is `get_phases_in_system()` (not `get_phase_names()` or `get_phases()`)
- `PhaseComposition` class is **not available** in the `tc_python` namespace

- All three DICTRA geometries confirmed: planar, cylindrical, spherical
- All required API classes confirmed: `Region`, `CalculatedGrid`, `CompositionProfile`, `ElementProfile`, `BoundaryCondition`, `Unit`, all 3 solvers, `SimulationTime`

10.4.2 Phase 3–6: Initial Failures and Script Bug Fixes

The initial run (March 14 AM) failed in Phases 3–6 due to three script bugs: wrong phase enumeration methods, empty phases list preventing pair selection, and an undefined `calc_succeeded` variable. All three were fixed, and LIQUID phase support at 1800 K was added.

10.4.3 Re-Run Results (March 14 PM: Fixed Script)

After fixing the script bugs, Phases 1–2 reproduced identically. Phase 3 reached the actual diffusion calculation but failed with a **new error**: the `Region` object does not specify which thermodynamic phase is the diffusion matrix.

Three API patterns were attempted:

1. **Pattern A** (CompositionProfile + ElementProfile): `IndexOutOfBoundsException: Index 0 out of bounds for length 0`. The DICTRA engine's internal phase list is empty because no phase was assigned to the Region.
2. **Pattern B** (PhaseComposition API): `'Region' object has no attribute 'add_phase_composition'`. This method does not exist in the installed TC-Python version.
3. **Pattern C** (minimal region): `All regions must have one or more phases`. TC-Python's own validation confirms the missing phase specification.

Phases 4–6 were skipped because Phase 3 failed. A subsequent run probed `dir(Region(...))` and discovered 8 methods, including the solution: `Region.add_phase(phase_name)`. With this call added, Phase 3 succeeded: Cu diffusion in FCC_A1 at 1200 K for 3600 s returned a 50-point composition profile (0.0000–0.5000 wt% Cu). The script was updated to add `.add_phase()` to all Phases 4–6, with temperature-dependent phase selection (LIQUID above 1700 K, FCC_A1 below).

10.4.4 Run 3 Results: Phases 4–6 (March 14, 6:42 PM)

With the `.add_phase()` fix applied to all phases, the full 6-phase test completed in ∼30 minutes. Phases 1–3 reproduced prior results. Phases 4–6 produced new data:

Phase 4: Cu Diffusion Coefficients vs Temperature. The script ran short (1 s) diffusion simulations at 8 temperatures and estimated D_{Cu} from profile displacement:

Temperature	Phase	D_{Cu} (m^2/s)
1073–1673 K	FCC_A1	$< 10^{-15}$ (below grid resolution)
1773 K	LIQUID	9.63×10^{-10}
1800 K	LIQUID	9.63×10^{-10}
1873 K	LIQUID	9.63×10^{-10}

The liquid-phase value $D_{\mathrm{Cu}} \approx 10^{-9}\,\mathrm{m^2/s}$ independently confirms the literature value [9]. The solid-state values appear as zero because 1 s simulation time is too short to resolve FCC diffusion ($D \sim 10^{-15}\,\mathrm{m^2/s}$ at 1073 K gives $\sqrt{2Dt} \approx 45\,\mathrm{nm}$, well below the 100 μm grid spacing).

Phase 5: Two-Region Steel/Slag Interface. Two tests modeled Cu transport across a metal/oxide boundary using TCOX14 + MOBFE8 (Cu-Fe-O):

- **Test A** (two LIQUID regions): **Failed.** The `CompositionProfile` only specified Cu; DICTRA requires all non-dependent elements (Cu, Fe with O as dependent). This is a script bug, not an API limitation.
- **Test B** (FCC_A1 | IONIC_LIQ#2 at 1800 K): **Succeeded.** Steel (0.3% Cu) and oxide slag ran as a diffusion couple for 3600 s. Cu redistributed: 36.2 wt% in the steel-side phase, 57.0 wt% in the slag-side phase. These are phase-internal compositions (not bulk averages), confirming DICTRA handles Cu partitioning across a metal/slag interface.

Phase 6: Boundary Conditions and Geometries. All four tests succeeded using TCFE13 + MOBFE8 (Cu-Fe) with FCC_A1 at 1200 K:

- **Fixed composition BC**: Cu = 0.50 wt% at left wall, 0.10 wt% at right wall after 3600 s. Gradient as expected.
- **Activity BC** ($a_{\mathrm{Cu}} = 0.001$ at right wall): Cu remained at 0.30 wt% everywhere. The imposed activity already matches equilibrium at this composition, so no driving force exists; the simulation is self-consistent but not Cu-removing.
- **Cylindrical geometry**: 50 μm radius, 600 s. Cu diffused inward from the surface.
- **Spherical geometry**: 50 μm radius sphere. Cu at center = 0.04 wt%, surface = 0.30 wt%, showing radial depletion toward the core.

10.4.5 Implications

Five of six DICTRA capability tests now succeed. The confirmed capabilities directly map to the Honda project needs:

- **Cu diffusion in liquid steel**: $D_{\mathrm{Cu}} = 9.63 \times 10^{-10}\,\mathrm{m^2/s}$ at 1800 K (matches literature)
- **Steel/slag interface**: two-region modeling with FCC_A1 and IONIC_LIQ works; Cu partitions across the boundary
- **Activity-based BC**: can impose a_{Cu} from oxide equilibrium at the steel surface
- **Spherical geometry**: can model Cu diffusion into oxide particles (relevant to powder-injection experiments)

The one remaining failure (Phase 5 Test A) is a script bug: the composition profile must specify all non-dependent elements, not just Cu. The DICTRA API itself is fully functional for all planned calculations.

11 $CUFE_2O_4$: DECOMPOSING THE DRIVING FORCE

$CuFe_2O_4$ has the strongest ΔG (-112 kJ at 1800 K), but its reaction involves both Cu capture *and* $Fe^{2+} \rightarrow Fe^{3+}$ oxidation. How much of the driving force comes from Cu affinity alone?

11.1 Method

We computed an alternative reaction using Fe_2O_3 instead of FeO:

$$Cu + Fe_2O_3 + \tfrac{1}{2}O_2 \rightarrow CuFe_2O_4$$

In this reaction, Fe is *already* at the +3 state, so any ΔG is purely from Cu incorporation into the spinel.

11.2 Result

- Cu-capture component: -45.7 kJ/mol (41% of total)
- Fe-oxidation component: -66.2 kJ/mol (59% of total)
- **Cu-capture alone is independently favorable**

Even without the Fe oxidation bonus, Cu incorporation into the spinel is thermodynamically driven. After activity correction ($n_{Cu} = 1$, penalty $= +54.9$ kJ), the Cu-capture component becomes $-45.7 + 54.9 = +9.2$ kJ, which is marginal. But the full reaction still sits at -57.0 kJ because the Fe oxidation energy is "free" (Fe is already being oxidized in the steelmaking slag).

12 V_2O_5: TWO PRODUCTS, TWO STORIES

V_2O_5 can form two different Cu compounds:

1. **$Cu_3V_2O_8$** ($n_{Cu} = 3$): Pure $\Delta G = -109$ kJ, but activity-corrected $\Delta G_{eff} = +55.6$ kJ. **Unfavorable** under steelmaking dilution.
2. **CuV_2O_6** ($n_{Cu} = 1$): Pure $\Delta G = -45$ kJ, activity-corrected $\Delta G_{eff} = +9.9$ kJ. **Uncertain**: near zero, within γ_{Cu} sensitivity range.

Our initial analysis only considered $Cu_3V_2O_8$ (ranked #2 by pure ΔG) and concluded V_2O_5 "fails." This was premature; the single-Cu product CuV_2O_6 is much more favorable under dilute conditions.

Lesson: Check All Stoichiometries

Under dilute-Cu conditions, multi-Cu products are heavily penalized. Always check whether the parent oxide can form a 1:1 Cu product. The "top 6 by pure ΔG" filter misses CuV_2O_6 because the 3-Cu product ranked higher. Selection criteria must account for the conditions under which the reaction will actually occur.

Bureau of Mines researchers tested V_2O_5 via oxide injection into molten steel and found it more effective than Al_2O_3 and MnO_2, but results remained far below commercially viable levels; slag Cu concentrations would need to increase 20–80× for economic feasibility [12]. At higher Cu concentrations and lower temperatures, both V_2O_5 products could become thermodynamically favorable.

13 TOWARD EXPERIMENTAL VALIDATION

13.1 Recommended Candidates

Based on the activity-corrected analysis:

1. **FeO/Fe_2O_3** (forming $CuFe_2O_4$ spinel): $\Delta G_{eff} = -57$ kJ. Robust across all γ_{Cu} values. Already present in steelmaking slag. **Top pick.**

2. **MnO** (forming $CuMn_2O_4$ spinel): $\Delta G_{\text{eff}} = -9.5$ kJ. Marginal but favorable. Also present in steelmaking slag. Slag basicity ($MnO:SiO_2$ ratio) can be tuned.

3. **V_2O_5** (forming CuV_2O_6): $\Delta G_{\text{eff}} = +10$ kJ (uncertain). Bureau of Mines tested V_2O_5 injection (more effective than Al_2O_3/MnO_2 but still far below viable [12]). Worth testing at higher Cu levels where activity penalty is smaller.

13.2 Mass Balance Estimates

A local calculator (`mass_balance_calculator.py`) computes stoichiometric oxide requirements for each candidate. For a 0.5 kg steel melt going from 0.30 to 0.10 wt% Cu (1.0 g Cu to remove):

Table 5: Oxide Requirements for 0.5 kg Steel, 0.3% → 0.1% Cu

Oxide	Product	Stoich (g)	3× (g)	wt%
Fe_2O_3	$CuFe_2O_4$	2.51	7.54	1.51
V_2O_5	$Cu_3V_2O_8$	0.95	2.86	0.57
MnO	$CuMn_2O_4$	2.23	6.70	1.34
SiO_2	Cu_2SiO_4	0.47	1.42	0.28
Al_2O_3	$CuAl_2O_4$	1.60	4.81	0.96

All oxide doses are small (<2 wt% at 3× excess), confirming that material cost is not a constraint. V_2O_5 is the most efficient per gram (3 Cu captured per mol oxide), but Fe_2O_3 has the strongest thermodynamic driving force.

Diffusion is Not the Bottleneck

$D_{\text{Cu}} = 9.63 \times 10^{-10}\,\text{m}^2/\text{s}$ in liquid Fe at 1800 K (from DICTRA Phase 4). For a 100 µm particle, the diffusion time is $r^2/2D \approx 5$ s. Cu reaches any particle within seconds. The rate-limiting step will be the chemical reaction at the particle surface, not transport through the melt.

Particle size trade-off: Smaller particles ($R = 10$–50 µm) give orders of magnitude more surface area per gram but dissolve faster into slag. At $R = 100$ µm, 7.5 g of Fe_2O_3 provides 3.4×10^5 particles with 432 cm^2 total surface area.

13.3 Cu Removal Rate Prediction (DICTRA)

A spherical particle model (`cu_removal_rate.py`) simulates the Cu depletion shell around an oxide particle in liquid steel. The setup:

- Spherical geometry: 5 mm steel shell around particle (center = particle surface)
- Left BC: fixed composition, Cu = 0.01 wt% at particle surface (oxide equilibrium sink)
- Right BC: closed system (far-field bulk steel at 0.30 wt% Cu)
- Sweep: 5 particle radii (25–500 µm) × 4 contact times (1–30 min)
- Database: TCFE13 + MOBFE8, LIQUID phase at 1800 K

Results (20/20 calculations succeeded): Each particle creates a Cu-depleted zone extending ∼250–500 µm into the surrounding steel. Cu captured per particle ranges from 2.2×10^{-3} mg (25 μm, 1 min) to 8.9×10^{-2} mg (500 μm, 30 min). Larger particles capture more per particle, but smaller particles provide orders of magnitude more surface area per gram of oxide.

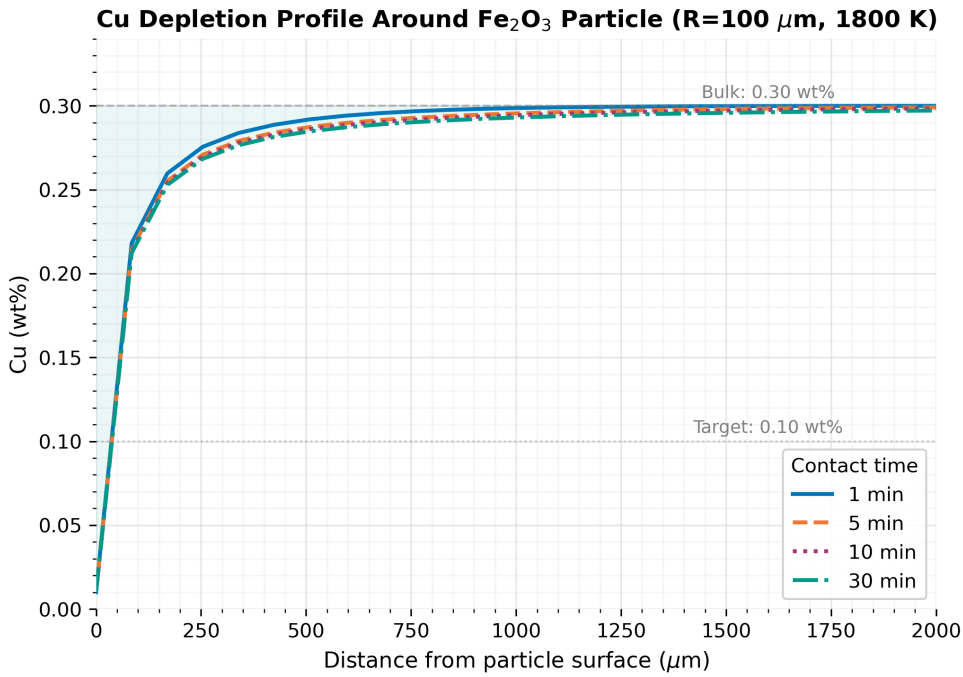

Figure 10: Cu depletion profile around a 100 µm Fe_2O_3 particle in liquid steel at 1800 K. The oxide surface (left) maintains Cu ≈ 0.01 wt%, creating a concentration gradient that draws Cu from the bulk (0.30 wt%). Longer contact times widen the depleted zone. Generated by `plot_cu_removal_rate.py`.

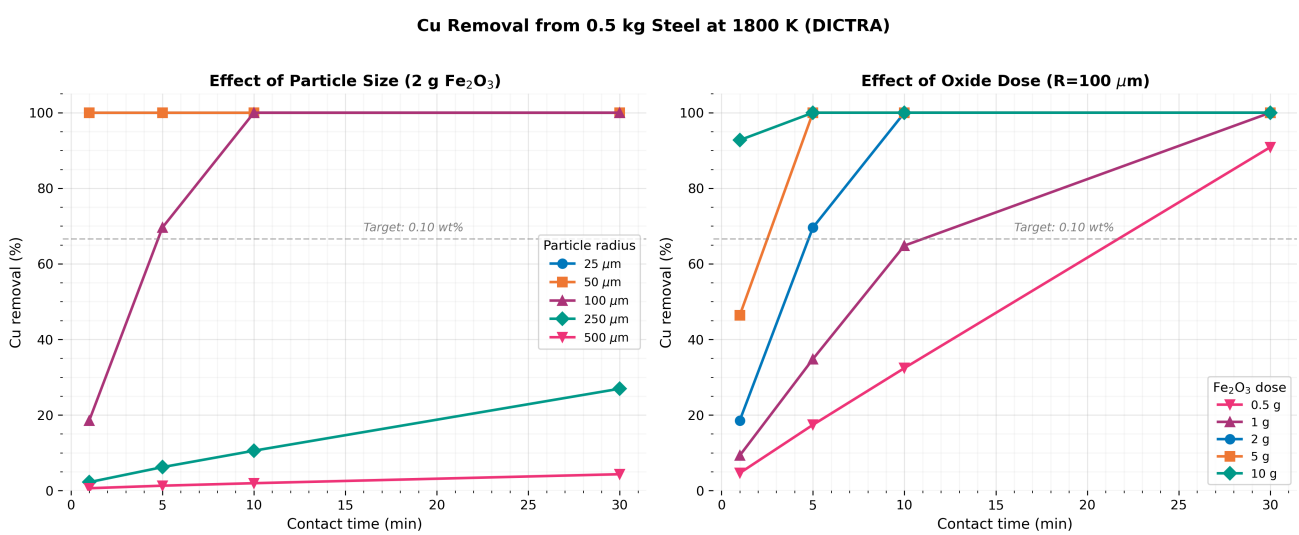

Figure 11: System-scale Cu removal from 0.5 kg steel at 1800 K. Left: effect of particle size at fixed 2 g Fe_2O_3 dose. Right: effect of oxide dose at fixed $R = 100$ µm. Dashed line marks the 0.10 wt% Cu target (67% removal). Generated by `plot_cu_removal_rate.py`.

Activity BC vs. Fixed Composition BC

The original script used an activity boundary condition ($a_{\mathrm{Cu}} = 0.001$) at the particle surface. This caused DICTRA to drive Cu *toward* the surface (100 wt% Cu) instead of away from it. Root cause: the Fe-Cu liquid miscibility gap creates two compositions with the same activity. DICTRA picked the Cu-rich root (∼100% Cu) instead of the Fe-rich root (∼0.01% Cu). Switching to a fixed composition BC (Cu = 0.01 wt%) eliminated the ambiguity. Lesson: use activity BCs only when the activity-composition mapping is monotonic.

13.4 Experiment Plan

Fontana Lab induction furnace with graphite crucible inside Al_2O_3 containment. Key variables:

- Steel: recycled scrap from Honda (XRF-characterized) or CDME
- Slag: 1–2 candidate oxides, possibly with Na_2S or CaF_2 flux
- Oxide dose: 1–2 wt% (from mass balance, Table 5)
- Temperature: 1500 °C–1600 °C
- Analysis: XRF (OES is currently down)
- Timeline: oxide ordering by April, experiments May–June

14 TECHNICAL PITFALLS AND LESSONS LEARNED

1. **GM is per mole of atoms, not formula units.** Multiply by atoms/formula. Getting this wrong gives energies off by 5–7×.
2. **O_2 reference = 2× GM(pure O).** TC-Python treats O as atomic, not molecular.
3. **Set $N - 1$ mole fractions.** The last element fills the remainder. Setting all N overconstrains the system.
4. **Empty element list = file existence check only.** Does not verify the database can load or has the needed elements/phases.
5. **Kinetic System API ≠ thermodynamic System API.** The object from the kinetic database selector has different methods (`get_phases_in_system()` not `get_phase_names()`).
6. **Citrix clipboard corrupts binary files.** Always transfer via ZIP download from GitHub, never copy-paste.
7. **"Top N by pure ΔG" is a bad filter for dilute systems.** Multi-Cu products rank high by pure ΔG but get crushed by the activity penalty. Must select candidates under actual conditions.
8. **SSUB3 is unreliable for complex oxides.** B_2O_3 from SSUB3 was 145% off literature values. Only use SSUB3 as a last resort, with verification.
9. **Flat a_{Cu} is not a bug, it's the Gibbs Phase Rule.** In a 2-phase field at fixed T, P, activity is invariant along the tie line. Sweeping overall composition changes phase amounts, not phase compositions.
10. **Dr. Zhang's FeO skepticism was about the binary reaction.** He was correct that Cu cannot reduce FeO. The ternary spinel reaction ($Cu + 2FeO + O_2 \rightarrow CuFe_2O_4$) is a different mechanism that he himself asked us to check.

15 GLOSSARY

CALPHAD Calculation of Phase Diagrams — computational thermodynamics method

DICTRA Diffusion Controlled Transformations — 1D multicomponent diffusion solver

TCOX14 Thermo-Calc oxide thermodynamic database (version 14)

TCFE13 Thermo-Calc steel thermodynamic database (version 13)

MOBFE8 Thermo-Calc steel mobility database (version 8)

GM Gibbs energy per mole of atoms (TC-Python output)

IONIC_LIQ Two-sublattice ionic liquid model for oxide melts

$a_{\mathbf{Cu}}$ Activity of Cu — effective thermodynamic concentration

$\gamma_{\mathbf{Cu}}$ Raoultian activity coefficient of Cu in liquid Fe

Hot shortness Surface cracking during hot working caused by Cu grain boundary segregation

Basicity Ratio of basic to acidic oxides in slag; controls activity coefficients

OLI Software for aqueous electrolyte thermodynamics (uses MSE model)

MQM Modified Quasichemical Model (FactSage's slag modeling approach)

SER Stable Element Reference (enthalpy reference state in CALPHAD)

A CHRONOLOGICAL LAB ENTRIES

This section records work in chronological order, showing how the project evolved from initial scoping through thermodynamic screening to kinetics exploration.

A.1 January 23–26, 2026: Project Kickoff

Jan 23: Project selected (Honda CALPHAD Cu removal). Initial folder structure created.

Jan 26: Kickoff meeting with Prof. Luo and Dr. Zhang via Zoom (10:28–10:54 AM). Established project scope: use CALPHAD to screen oxide materials for Cu removal from recycled steel. HW1 Meeting Report written and formatted with OSU-branded LaTeX template.

A.2 January 31 – February 4, 2026: First Calculations

Jan 31: Created pyCALPHAD simulation folder. Built Ellingham diagram analysis (`cu_ceramic_affinity.py`) using literature data. Key finding: Cu oxides are least stable, so direct oxide reduction will not work. Created GitHub repo: `dimascad/honda-calphad`.

Feb 1: Built interactive Marimo notebook (`cu_ceramic_thermodynamics.py`) with temperature slider and oxide selector. Deployed to Molab. Created `DOCUMENTATION.tex` and `THERMOCALC_GUIDE.tex`.

Feb 4: **First VM run.** Ran `extract_oxide_gibbs.py` on OSU ETS VM using Thermo-Calc TCOX14 database. Fixed three critical bugs:

1. GM normalization: per-atom → per-formula (off by 5–7×)
2. O_2 reference: $G_{O_2} = 2 \times \texttt{GM}$ (was using 1×)
3. Two-phase mixture: was reading mixture energy instead of single-phase

Successfully extracted ΔG_f for 7 oxides × 31 temperatures. Created Ellingham diagram with real TCOX14 data.

A.3 February 12, 2026: Dr. Zhang Meeting: Direction Shift

Advisor meeting (2:00–2:40 PM, 2165 Fontana Lab) with Dr. Zhang:

> "We want to use the oxide to capture the copper. Last time, [Al_2O_3] worked because of the spinel reaction."

Key outcomes:

- Dr. Zhang **confirmed** Ellingham results are correct: Cu_2O is least stable
- Al_2O_3 tested experimentally last year: some Cu reduction but Na_2S salt was better (95%, per Zhang)
- **New direction**: Model ternary compound formation ($Cu + MO_x + O_2 \rightarrow CuMO_y$), not binary exchange
- Screening table required: oxide name, density, toxicity, reaction ΔG, cost
- Timeline: 2 weeks for calculations → meet end of March → Honda call after

Also submitted HW2 Project Charter (Feb 11).

A.4 March 12, 2026: Screening Pipeline Complete

Full-day session. Expanded from 7 to 18 oxides. Completed the screening table with all 10 candidate oxides having computed ΔG_{rxn} values, then ran the 18-oxide TCOX14 extraction on the VM (15 direct, 2 from SSUB3 fallback for PbO and CeO_2, plus partial B_2O_3). Built `Oxide_Screening_Complete.xlsx` with 5 sheets and cell formulas tracing the full GM $\rightarrow$ kJ/mol O_2 normalization pipeline.

Swept the entire periodic table: 47 oxides evaluated, 7 "moonshots" selected (NiO, CoO, PbO, B_2O_3, V_2O_5, La_2O_3, CeO_2). PbO turned out to be an interesting edge case: $\Delta G_{\text{rxn}} = -34.7$ kJ at 727 °C (Cu CAN reduce it at low T) but +136.2 kJ at 1527 °C. SSUB3 reliability was mixed: B_2O_3 was 145% off literature values (rejected), CeO_2 was 19% off (accepted with caveat).

A.5 March 13, 2026: Ternary Breakthrough

Dr. Zhang meeting (11 AM): presented 18-oxide screening. Dr. Zhang said "good work" and directed the ternary modeling effort.

Morning: Created `extract_ternary_reactions.py` and `compute_ternary_dG.py`. Pushed to GitHub.

Evening, VM extraction: Ran ternary extraction on OSU VM. **ALL 16 REACTIONS FAVORABLE.** Every Cu + MO_x + $O_2 \rightarrow CuMO_y$ reaction has $\Delta G < 0$ at 1800 K. Top 3: $CuFe_2O_4$ (−112 kJ), $Cu_3V_2O_8$ (−109 kJ), $CuMn_2O_4$ (−64 kJ).

Late evening, validation suite: Ran all 5 VM scripts ($CuFe_2O_4$ decomposition, Cu activity, ΔG vs. T, slag basicity, phase maps) and all 6 local post-processing scripts. Verification: 136 overlapping ΔG points matched to 0.00 kJ.

Key discoveries:

- No crystalline ternary phase survives at 1800 K; Cu capture = slag dissolution
- $CuFe_2O_4$: 41% Cu-capture, 59% Fe-oxidation. Cu-capture alone is favorable.
- Cu activity is flat in 2-phase fields (Gibbs Phase Rule, not a code bug)
- V_2O_5 has **two** products: $Cu_3V_2O_8$ (unfavorable dilute) and CuV_2O_6 (marginal)

A.6 March 14, 2026: DICTRA Kinetics Testing

Pivoted from thermodynamics ("does it react?") to kinetics ("how fast?").

DICTRA test script (`test_dictra_diffusion.py`): 6-phase script probing the TC-Python diffusion API on the OSU VM:

1. Database compatibility matrix: 312 combinations (4 thermo × 13 mobility × 6 element sets)
2. System API exploration: discovered `get_phases_in_system()` (not `get_phase_names()`)
3. Single-region diffusion: Cu gradient in FCC austenite
4. D_{Cu} vs. T: diffusion coefficients at 8 temperatures
5. Two-region interface: steel | slag boundary
6. Boundary conditions: activity-based, cylindrical, spherical geometry

Results (∼1 hour runtime, 150 KB log):

- **Phase 1–2 SUCCEEDED:** 232/312 combos loaded; 40 unique thermo+mobility pairs support diffusion. Correct API method discovered: `get_phases_in_system()`.

- **Phase 3–6 FAILED:** Script bugs (wrong phase enumeration methods, empty phases list, undefined variable). All three fixed.
- MOBFE9, MOBOX1/2: not found on VM. MOBFE8/7/6/5 all work.
- `PhaseComposition` class not in `tc_python` namespace.
- Log archived: `data/tcpython/logs/2026-03-14_dictra_capability_test.rtf`

Script fixed and ready for re-run. Added LIQUID phase support at 1800 K.
Run 3 results (6:42 PM, 30 min runtime):

- **Phase 3 confirmed:** Cu diffusion in FCC_A1 at 1200 K, 50-point profile.
- **Phase 4:** $D_{\mathrm{Cu}} = 9.63 \times 10^{-10}\,\mathrm{m^2/s}$ in LIQUID at 1773–1873 K. Independently confirms Hino & Ito (2010) literature value ($\sim 10^{-9}$). Solid-state values below grid resolution at 1 s sim time.
- **Phase 5 Test B:** Steel/slag interface (FCC_A1 | IONIC_LIQ#2) at 1800 K ran successfully. Cu partitions across the boundary. Test A failed due to incomplete composition profile (script bug, not API).
- **Phase 6:** All 4 tests passed: fixed-composition BC, activity BC ($a_{\mathrm{Cu}} = 0.001$), cylindrical (50 μm radius), spherical (center depleted to 0.04 wt% from 0.30 wt% surface).
- Log archived: `data/tcpython/logs/2026-03-14_dictra_run3_642pm.rtf`

DICTRA Validates Literature and Enables Kinetic Modeling

The computed $D_{\mathrm{Cu}} = 9.63 \times 10^{-10}\,\mathrm{m^2/s}$ at 1800 K matches the Hino & Ito (2010) reference value of $\sim 10^{-9}\,\mathrm{m^2/s}$ for impurity diffusion in liquid iron. With steel/slag interface modeling, activity-based BCs, and spherical geometry all confirmed working, the DICTRA API can now model Cu transport from liquid steel into oxide particles: the core kinetic question for the Honda project.

A.7 March 14, 2026: Unified Workbook

Compiled all screening and validation data into `Cu_Removal_Unified.xlsx` (15 tabs). Analysis tabs use Excel cell formulas that reference the raw data tabs, so every number is traceable: click any cell to see where it came from. Tabs are color-coded: blue for analysis, green for derived results, gray for raw TC-Python output.

Excel charts added (live, update with data changes):

- $CuFe_2O_4$ DeepDive: 3-series line chart (total ΔG, alternative, Fe-oxidation contribution)
- Activity_and_Slag: 7-product sensitivity chart (ΔG_{eff} vs. γ_{Cu}), colors matched to matplotlib figures
- Normalization: horizontal bar chart showing ΔG° per mol O_2 at 1800 K for all 17 binary oxides

Data provenance table on Dashboard enhanced with "Generating Script" column containing clickable GitHub hyperlinks to each script's source code.

Activity Coefficient Verification

$\gamma_{\mathrm{Cu}} = 8.5$ confirmed from Hino & Ito (2010), *Thermodynamic Data for Steelmaking*, Tohoku University Press. Alternative: Sigworth & Elliott (1974, *Metal Science*, 755+ citations) give $\sim$10. Literature range 5–13. Both citations verified via web search.

A.8 March 14, 2026: Experiment Design Tools

Built two tools connecting thermodynamic screening to experiment design:

Mass balance calculator (`screening/mass_balance_calculator.py`): computes stoichiometric oxide requirements for all 5 top candidates. For 0.5 kg steel going 0.30% → 0.10% Cu, Fe_2O_3 needs only 7.5 g at 3× excess (1.5 wt%). V_2O_5 is most efficient at 2.9 g (0.57 wt%) because each mol captures 3 Cu. Particle size sweep shows $R = 50$–$100\,\mu m$ is optimal: diffusion time $r^2/2D \approx 1$–$5\,s$ (liquid-phase transport is essentially instantaneous vs. experiment hold times of 10–30 min).

Cu removal rate script (`simulations/tcpython/cu_removal_rate.py`): DICTRA spherical particle model. A 5 mm steel shell surrounds the particle with Cu = 0.01 wt% at the surface (fixed composition BC) and closed far-field BC. Sweeps 5 radii × 4 contact times = 20 calculations. Successfully executed on VM (see lab entry below).

Accuracy audit of all project numbers: 25 claims checked across thermodynamic values, literature references, and internal consistency. Zero substantive errors found. Fixed one broken URL in `capstone.bib` (911Metallurgist page had moved).

CLAUDE.md pruned: 472 → 323 lines. Session log history (Jan 23 – Mar 12) archived to `SESSION_LOG.md`. Stale action items removed.

A.9 March 14, 2026: DICTRA Cu Removal Rate — 20/20 Success

Ran `cu_removal_rate.py` on the OSU VM. Four iterations to get it right:

Run 1: `TCPython()` returns a wrapper, not the session. Fixed by using `TCPython().__enter__()` to get the actual API object.

Run 2: Script selected FCC_A1 as diffusion phase at 1800 K. Steel is liquid above ∼1723 K. Phase selection logic had a bug: after finding LIQUID, a fallback check `if diff_phase == "LIQUID"` was always true, overwriting with FCC_A1. Fixed by using `None` as sentinel.

Run 3: Activity BC ($a_{Cu} = 0.001$) drove Cu to 100 wt% at the particle surface instead of depleting it. All 20 calculations returned negative `cu_captured_mg`. Root cause: Fe-Cu liquid miscibility gap. At 1800 K, two liquid compositions satisfy any given a_{Cu}: one Fe-rich (∼0.01% Cu) and one Cu-rich (∼100% Cu). DICTRA picked the wrong root.

Run 4 (success): Replaced activity BC with fixed composition BC (Cu = 0.01 wt% at surface). All 20/20 calculations completed in 35 seconds. Cu at surface = 0.01 wt% (depleted from 0.30%), positive capture values, depletion zones extending 250–500 µm into the steel. Output CSVs: `cu_removal_rate_profiles.csv` (1200 rows) and `cu_removal_rate_summary.csv` (20 rows).

Figures generated: Cu depletion profiles (Fig. 10), system-scale removal predictions (Fig. 11), and Cu captured per particle vs. time.

File organization: 9 loose RTF terminal logs archived from capstone root into `data/tcpython/logs/` with consistent `YYYY-MM-DD_description.rtf` naming.

Miscibility Gap Traps in Boundary Conditions

The Fe-Cu binary has a well-known liquid miscibility gap. Activity is not monotonic across it: the same a_{Cu} maps to two compositions. DICTRA's root-finder has no way to know which one you want. Fixed composition BCs are always unambiguous. This is a general lesson: use activity BCs only when the activity-composition curve is monotonic in the phase of interest.

B SCRIPT INDEX

All scripts in the `honda-calphad` repository, organized by location and purpose.

B.1 VM Scripts (`simulations/tcpython/`)

These run on the OSU ETS virtual machine where Thermo-Calc and TC-Python are installed.

Table 6: TC-Python Scripts for OSU VM Execution

Script	Purpose
`extract_oxide_gibbs.py`	Extract ΔG_f for 18 oxides from TCOX14 (Ellingham data). 500–2000K, 50K steps.
`extract_ssub3_fallback.py`	SSUB3 fallback for PbO, CeO_2, B_2O_3 (not in TCOX14).
`extract_ternary_reactions.py`	Ternary reaction GM extraction: $Cu + MO_x + O_2 \rightarrow CuMO_y$. 15 systems, 18 products.
`cufe2o4_alternative_reaction.py`	Decompose $CuFe_2O_4$ ΔG into Cu-capture vs. Fe-oxidation components.
`cu_activity_vs_oxide.py`	Sweep a_{Cu} vs. composition at 1800 K for 4 ternary systems.
`dG_vs_T_top6.py`	Fine ΔG vs. T curves (25K steps) for top 6 candidates with phase tracking.
`slag_composition_effects.py`	Quaternary slag basicity effects on a_{Cu} (Cu-Mn-Si-O, Cu-Al-Si-O).
`ternary_phase_map_1800K.py`	20×20 composition grid phase mapping at 1800 K.
`test_dictra_diffusion.py`	DICTRA capability test: database matrix, diffusion API, boundary conditions.
`check_databases.py`	Probe which TC databases are installed on the VM.
`check_mobox_tcox.py`	Test MOBOX/TCOX mobility database availability.
`check_vm_capabilities.py`	General VM environment probe (Python version, packages).
`verify_oxide_data.py`	Cross-check extracted oxide data against literature values.
`test_tcox_api.py`	Early API exploration script for TCOX14 capabilities.
`cu_al_o_phase_stability.py`	Cu-Al-O phase stability exploration.
`cu_activity_combined_db.py`	Attempt Cu activity with combined TCFE+TCOX database.
`cu_removal_rate.py`	DICTRA spherical particle model: Cu depletion around oxide particle in liquid steel. 5 radii × 4 times.

B.2 Local Post-Processing Scripts (`screening/`)

These run locally on the Mac after copying CSVs from the VM.

Table 7: Local Post-Processing and Analysis Scripts

Script	Purpose
`compute_ternary_dG.py`	Compute ΔG for all ternary reactions from raw GM data.
`compute_reaction_dG.py`	Compute binary reaction ΔG (Ellingham analysis).
`compute_activity_corrected_dG.py`	Apply dilute Cu activity correction ($\gamma_{\mathrm{Cu}} \cdot x_{\mathrm{Cu}}$).
`analyze_cufe2o4_decomposition.py`	Split $CuFe_2O_4$ driving force into Cu-capture vs. Fe-oxidation.
`plot_cu_activity.py`	Figure: a_{Cu} vs. oxide fraction for 4 systems.
`plot_dG_vs_T.py`	Figure: ΔG vs. T for top 6 candidates + verification.
`plot_ternary_phase.py`	Figures: ternary composition diagrams colored by dominant phase.
`plot_slag_effects.py`	Figure: a_{Cu} vs. slag basicity ratio.
`update_screening_table.py`	Update screening CSV with computed values.
`compute_activity_corrected_dG.py`	Activity-corrected ΔG for dilute Cu ($X_{\mathrm{Cu}} = 0.003$, $\gamma_{\mathrm{Cu}} = 8.5$).
`mass_balance_calculator.py`	Stoichiometric oxide requirements, particle counts, surface area, diffusion time estimates.
`plot_cu_removal_rate.py`	Figures: Cu depletion profiles, per-particle capture, and system-scale removal from DICTRA data.
`build_unified_xlsx.py`	Build `Cu_Removal_Unified.xlsx`: 16-tab workbook with formula-driven analysis tabs and raw data tabs.

B.3 Marimo Notebooks (`simulations/notebooks/`)

Interactive Python notebooks deployed to Molab for visualization.

Table 8: Marimo Interactive Notebooks

Script	Purpose
`ellingham_diagram.py`	Interactive Ellingham diagram with oxide selector. Deployed at Molab.
`cu_ceramic_affinity.py`	Oxide stability comparison using literature data.
`cu_o_visualization.py`	Cu-O system visualization.
`cu_o_pycalphad.py`	pyCALPHAD-based Cu-O computation.
`compute_cu_o_data.py`	Data generation for Cu-O plots.
`pycalphad_cu_fe_example.py`	pyCALPHAD demonstration with Cu-Fe binary system.

C DATA FILES

All raw data is stored in `data/tcpython/raw/`. Each CSV is produced by one VM script and consumed by one or more local post-processing scripts.

Table 9: Raw Data Files from TC-Python VM Runs

File	Size	Source Script
`oxide_gibbs_energies.csv`	54 kB	`extract_oxide_gibbs.py`
`ternary_reaction_energies.csv`	92 kB	`extract_ternary_reactions.py`
`dG_vs_T_top6.csv`	57 kB	`dG_vs_T_top6.py`
`cufe2o4_alternative_reaction.csv`	5 kB	`cufe2o4_alternative_reaction.py`
`cu_activity_vs_oxide.csv`	7 kB	`cu_activity_vs_oxide.py`
`slag_composition_effects.csv`	4 kB	`slag_composition_effects.py`
`ternary_phase_map_1800K.csv`	106 kB	`ternary_phase_map_1800K.py`
`cu_removal_rate_summary.csv`	2 kB	`cu_removal_rate.py`
`cu_removal_rate_profiles.csv`	85 kB	`cu_removal_rate.py`

All figures are stored in `figures/` as both PNG (for this document) and PDF (for vector-quality printing). Every figure has a corresponding script in the "Source Script" column of Table 7.

D SELECTED SCRIPTS

This appendix includes the scripts needed to reproduce the key results in this notebook. Only scripts that generate primary data or perform critical analysis are included; plotting scripts and Excel builders are omitted (they are in the GitHub repository). Scripts are printed in order of the analysis pipeline: data extraction first, then post-processing.

Each script has a brief introduction explaining what it does, what database it uses, where it runs, and which section of the notebook uses its output.

D.1 extract_oxide_gibbs.py

Runs on: OSU ETS VM (requires TC-Python + TCOX14 [4])
Runtime: ∼5 minutes
Output: `data/tcpython/raw/oxide_gibbs_energies.csv`
Used in: Section 1 (Ellingham diagram), Normalization tab of unified workbook

Extracts Gibbs energy of formation (ΔG_f) for 18 binary oxides from TCOX14 over 500–2000 K in 50 K steps. For each oxide, it computes: (1) G of the oxide phase at its stoichiometric composition, (2) G of the pure metal reference, and (3) G of O_2 gas. The formation energy is $\Delta G_f = G_{\text{oxide}} - n\,G_{\text{metal}} - m\,G_{O_2}$, normalized per mole O_2 for the Ellingham diagram. This is the foundation of the entire screening pipeline.

```python
#!/usr/bin/env python3
"""
Extract Gibbs energy of FORMATION for oxides from Thermo-Calc TCOX14.

Method:
1. For each oxide, get G of the oxide phase
2. Get G of the metal reference state
3. Get G of O2 gas reference
4. Calculate: dG_f = G(oxide) - n*G(metal) - m*G(O2)
5. Normalize per mole O2 for Ellingham diagram

Output: ../../data/tcpython/raw/oxide_gibbs_energies.csv
"""

import csv
from pathlib import Path
from tc_python import *

# ============================================================================
# Configuration
# ============================================================================
T_MIN = 500
T_MAX = 2000
T_STEP = 50

SCRIPT_DIR = Path(__file__).parent
OUTPUT_DIR = SCRIPT_DIR.parent.parent / "data" / "tcpython" / "raw"
OUTPUT_FILE = OUTPUT_DIR / "oxide_gibbs_energies.csv"

# ============================================================================
# Oxide definitions
# For dG_f calculation: oxide from metal + O2
# Reaction per mole O2: (metal_coeff)*M + O2 -> (oxide_coeff)*MxOy
# ============================================================================
OXIDES = {
    # name: elements, X_O for oxide, metal_coeff per O2, oxide_coeff per O2, phase_patterns
    # phase_patterns: TC phase names to search for (in priority order)
    # atoms_per_formula: number of atoms in one formula unit (for GM normalization)
    "Cu2O": {
        "elements": ["CU", "O"],
        "X_O": 0.333,  # Cu2O = 2Cu + 1O, X_O = 1/3
        "metal_per_O2": 4,     # 4Cu + O2 -> 2Cu2O
        "oxide_per_O2": 2,
```

```
        "phase_patterns": ["CUPRITE", "CU2O"],
        "atoms_per_formula": 3,  # Cu2O = 2+1 = 3 atoms
    },
    "CuO": {
        "elements": ["CU", "O"],
        "X_O": 0.500,  # CuO = 1Cu + 1O, X_O = 1/2
        "metal_per_O2": 2,     # 2Cu + O2 -> 2CuO
        "oxide_per_O2": 2,
        "phase_patterns": ["CUO", "TENORITE"],
        "atoms_per_formula": 2,  # CuO = 1+1 = 2 atoms
    },
    "Al2O3": {
        "elements": ["AL", "O"],
        "X_O": 0.600,  # Al2O3 = 2Al + 3O, X_O = 3/5
        "metal_per_O2": 4/3,   # 4/3Al + O2 -> 2/3Al2O3
        "oxide_per_O2": 2/3,
        "phase_patterns": ["CORUNDUM", "AL2O3"],
        "atoms_per_formula": 5,  # Al2O3 = 2+3 = 5 atoms
    },
    "MgO": {
        "elements": ["MG", "O"],
        "X_O": 0.500,  # MgO = 1Mg + 1O
        "metal_per_O2": 2,     # 2Mg + O2 -> 2MgO
        "oxide_per_O2": 2,
        "phase_patterns": ["HALITE", "MGO", "PERICLASE"],
        "atoms_per_formula": 2,  # MgO = 1+1 = 2 atoms
    },
    "SiO2": {
        "elements": ["SI", "O"],
        "X_O": 0.667,  # SiO2 = 1Si + 2O
        "metal_per_O2": 1,     # Si + O2 -> SiO2
        "oxide_per_O2": 1,
        "phase_patterns": ["QUARTZ", "SIO2", "TRIDYMITE", "CRISTOBALITE"],
        "atoms_per_formula": 3,  # SiO2 = 1+2 = 3 atoms
    },
    "TiO2": {
        "elements": ["TI", "O"],
        "X_O": 0.667,  # TiO2 = 1Ti + 2O
        "metal_per_O2": 1,     # Ti + O2 -> TiO2
        "oxide_per_O2": 1,
        "phase_patterns": ["RUTILE", "TIO2", "ANATASE"],
        "atoms_per_formula": 3,  # TiO2 = 1+2 = 3 atoms
    },
    "FeO": {
        "elements": ["FE", "O"],
        "X_O": 0.500,  # FeO = 1Fe + 1O
        "metal_per_O2": 2,     # 2Fe + O2 -> 2FeO
        "oxide_per_O2": 2,
        "phase_patterns": ["HALITE", "FEO", "WUSTITE"],  # FeO is halite structure
        "atoms_per_formula": 2,  # FeO = 1+1 = 2 atoms
    },
    # --- NEW OXIDES (added Feb 2026 for expanded screening) ---
    "CaO": {
        "elements": ["CA", "O"],
        "X_O": 0.500,  # CaO = 1Ca + 1O
        "metal_per_O2": 2,     # 2Ca + O2 -> 2CaO
        "oxide_per_O2": 2,
        "phase_patterns": ["HALITE", "CAO", "LIME"],  # CaO has NaCl (halite) structure
        "atoms_per_formula": 2,
    },
    "ZrO2": {
        "elements": ["ZR", "O"],
        "X_O": 0.667,  # ZrO2 = 1Zr + 2O
        "metal_per_O2": 1,     # Zr + O2 -> ZrO2
        "oxide_per_O2": 1,
        "phase_patterns": ["BADDELEYITE", "ZRO2", "FLUORITE", "MONOCLINIC"],
        "atoms_per_formula": 3,
    },
    "Cr2O3": {
        "elements": ["CR", "O"],
```

```
        "X_O": 0.600,  # Cr2O3 = 2Cr + 3O, X_O = 3/5
        "metal_per_O2": 4/3,   # 4/3Cr + O2 -> 2/3Cr2O3
        "oxide_per_O2": 2/3,
        "phase_patterns": ["CORUNDUM", "CR2O3", "ESKOLAITE"],  # Same structure as Al2O3
        "atoms_per_formula": 5,
    },
    "MnO": {
        "elements": ["MN", "O"],
        "X_O": 0.500,  # MnO = 1Mn + 1O
        "metal_per_O2": 2,     # 2Mn + O2 -> 2MnO
        "oxide_per_O2": 2,
        "phase_patterns": ["HALITE", "MNO", "MANGANOSITE"],  # NaCl structure like MgO/FeO
        "atoms_per_formula": 2,
    },
    # --- MOONSHOT OXIDES (added Mar 2026 for expanded comparison) ---
    "NiO": {
        "elements": ["NI", "O"],
        "X_O": 0.500,  # NiO = 1Ni + 1O
        "metal_per_O2": 2,     # 2Ni + O2 -> 2NiO
        "oxide_per_O2": 2,
        "phase_patterns": ["HALITE", "NIO", "BUNSENITE"],  # NaCl structure
        "atoms_per_formula": 2,
    },
    "CoO": {
        "elements": ["CO", "O"],
        "X_O": 0.500,  # CoO = 1Co + 1O
        "metal_per_O2": 2,     # 2Co + O2 -> 2CoO
        "oxide_per_O2": 2,
        "phase_patterns": ["HALITE", "COO"],  # NaCl structure
        "atoms_per_formula": 2,
    },
    "PbO": {
        "elements": ["PB", "O"],
        "X_O": 0.500,  # PbO = 1Pb + 1O
        "metal_per_O2": 2,     # 2Pb + O2 -> 2PbO
        "oxide_per_O2": 2,
        "phase_patterns": ["LITHARGE", "PBO", "MASSICOT"],
        "atoms_per_formula": 2,
    },
    "B2O3": {
        "elements": ["B", "O"],
        "X_O": 0.600,  # B2O3 = 2B + 3O, X_O = 3/5
        "metal_per_O2": 4/3,   # 4/3B + O2 -> 2/3B2O3
        "oxide_per_O2": 2/3,
        "phase_patterns": ["B2O3", "BORON_OXIDE"],
        "atoms_per_formula": 5,  # B2O3 = 2+3 = 5 atoms
    },
    "V2O5": {
        "elements": ["V", "O"],
        "X_O": 5/7,    # V2O5 = 2V + 5O, X_O = 5/7
        "metal_per_O2": 4/5,   # 4/5V + O2 -> 2/5V2O5
        "oxide_per_O2": 2/5,
        "phase_patterns": ["V2O5", "SHCHERBINAITE"],
        "atoms_per_formula": 7,  # V2O5 = 2+5 = 7 atoms
    },
    "La2O3": {
        "elements": ["LA", "O"],
        "X_O": 0.600,  # La2O3 = 2La + 3O, X_O = 3/5
        "metal_per_O2": 4/3,   # 4/3La + O2 -> 2/3La2O3
        "oxide_per_O2": 2/3,
        "phase_patterns": ["LA2O3", "A_LA2O3", "C_LA2O3", "HEXAGONAL"],
        "atoms_per_formula": 5,  # La2O3 = 2+3 = 5 atoms
    },
    "CeO2": {
        "elements": ["CE", "O"],
        "X_O": 2/3,    # CeO2 = 1Ce + 2O, X_O = 2/3
        "metal_per_O2": 1,     # Ce + O2 -> CeO2
        "oxide_per_O2": 1,
        "phase_patterns": ["FLUORITE", "CEO2", "CERIANITE"],
        "atoms_per_formula": 3,  # CeO2 = 1+2 = 3 atoms
```

```
    },
}


def get_phase_gibbs(result, phase_patterns):
    """
    Get Gibbs energy of a specific phase matching one of the patterns.
    Returns (GM_value, phase_name) or (None, None) if not found.
    """
    stable_phases = result.get_stable_phases()
    for pattern in phase_patterns:
        for phase in stable_phases:
            if pattern.upper() in phase.upper():
                try:
                    gm = result.get_value_of(f"GM({phase})")
                    return gm, phase
                except:
                    pass
    return None, None


def main():
    print("=" * 70)
    print("TC-Python: Oxide Formation Energies for Ellingham Diagram")
    print("=" * 70)

    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    temperatures = list(range(T_MIN, T_MAX + 1, T_STEP))
    print(f"Temperature range: {T_MIN}-{T_MAX} K ({len(temperatures)} points)\n")

    results = {T: {"T_K": T, "T_C": T - 273.15} for T in temperatures}

    with TCPython() as session:
        print("Connected to Thermo-Calc")

        # First, get O2 gas reference energy at each temperature
        # Note: GM returns per mole of O atoms, so multiply by 2 for O2 molecule
        print("\n--- Getting O2 reference energies ---")
        G_O2_per_mol_O2 = {}
        try:
            o2_system = session.select_database_and_elements("TCOX14", ["O"]).get_system()
            for T in temperatures:
                calc = o2_system.with_single_equilibrium_calculation()
                calc.set_condition(ThermodynamicQuantity.temperature(), T)
                calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
                result = calc.calculate()
                # GM is per mole O atoms; multiply by 2 for per mole O2
                GM_O = result.get_value_of("GM")
                G_O2_per_mol_O2[T] = 2 * GM_O
            print(f"  O2 reference at 1000K: {G_O2_per_mol_O2.get(1000, 'N/A')} J/mol-O2")
        except Exception as e:
            print(f"  ERROR getting O2: {e}")
            print("  Using G_O2 = 0 (SER reference)")
            G_O2_per_mol_O2 = {T: 0 for T in temperatures}

        # Process each oxide
        for oxide_name, config in OXIDES.items():
            print(f"\n--- Processing {oxide_name} ---")
            elements = config["elements"]
            X_O = config["X_O"]
            metal = elements[0]

            try:
                system = session.select_database_and_elements("TCOX14", elements).get_system()

                # Get metal reference energy
                print(f"  Getting {metal} reference...")
                G_metal = {}
                for T in temperatures:
                    calc = system.with_single_equilibrium_calculation()
```

```python
                    calc.set_condition(ThermodynamicQuantity.temperature(), T)
                    calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
                    calc.set_condition(ThermodynamicQuantity.mole_fraction_of_a_component("O"), 0.0001)  # Nearly pure metal
                    result = calc.calculate()
                    G_metal[T] = result.get_value_of("GM")

                # Get oxide energy at stoichiometric composition
                print(f"  Getting {oxide_name} oxide phase...")
                phase_patterns = config["phase_patterns"]
                success = 0
                for T in temperatures:
                    try:
                        calc = system.with_single_equilibrium_calculation()
                        calc.set_condition(ThermodynamicQuantity.temperature(), T)
                        calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
                        calc.set_condition(ThermodynamicQuantity.mole_fraction_of_a_component("O"), X_O)
                        result = calc.calculate()

                        stable = result.get_stable_phases()
                        GM_system = result.get_value_of("GM")

                        # Get the INDIVIDUAL oxide phase Gibbs energy, not mixture
                        GM_oxide, oxide_phase = get_phase_gibbs(result, phase_patterns)

                        # Store raw data
                        results[T][f"GM_{oxide_name}"] = GM_oxide if GM_oxide else GM_system
                        results[T][f"GM_system_{oxide_name}"] = GM_system
                        results[T][f"G_metal_{oxide_name}"] = G_metal[T]
                        results[T][f"phases_{oxide_name}"] = ";".join(stable)
                        results[T][f"oxide_phase_{oxide_name}"] = oxide_phase if oxide_phase else "NOT_FOUND"

                        # Calculate dG_f per mole O2
                        metal_coeff = config["metal_per_O2"]
                        oxide_coeff = config["oxide_per_O2"]
                        atoms_per_formula = config["atoms_per_formula"]

                        # Use individual phase energy if available, else fall back to system
                        GM_for_calc = GM_oxide if GM_oxide else GM_system

                        # IMPORTANT: GM is per mole of ATOMS/SITES, not per formula unit
                        # Convert to per formula unit: G_formula = GM * atoms_per_formula
                        G_oxide_per_formula = GM_for_calc * atoms_per_formula

                        # dG per mole O2 = oxide_coeff*G(oxide) - metal_coeff*G(metal) - G(O2)
                        # G_metal is already per mole of metal atoms (which = per formula unit for elements)
                        dG_per_O2 = oxide_coeff * G_oxide_per_formula - metal_coeff * G_metal[T] - G_O2_per_mol_O2[T]
                        results[T][f"dG_{oxide_name}_per_O2"] = dG_per_O2

                        success += 1
                    except Exception as e:
                        results[T][f"GM_{oxide_name}"] = None
                        results[T][f"dG_{oxide_name}_per_O2"] = None
                        results[T][f"phases_{oxide_name}"] = f"Error: {e}"

                print(f"  Completed: {success}/{len(temperatures)} temperatures")

                # Sample output
                if 1000 in results and results[1000].get(f"dG_{oxide_name}_per_O2"):
                    dG = results[1000][f"dG_{oxide_name}_per_O2"]
                    phases = results[1000].get(f"phases_{oxide_name}", "")
                    used_phase = results[1000].get(f"oxide_phase_{oxide_name}", "")
                    print(f"  dG at 1000K: {dG/1000:.1f} kJ/mol O2")
                    print(f"  Stable phases: {phases}")
                    print(f"  Used for calc: {used_phase}")

```

```
            except Exception as e:
                print(f"  SYSTEM ERROR: {e}")

        # Write CSV
        print(f"\n{'='*70}")
        print(f"Writing to {OUTPUT_FILE}")

        fieldnames = ["T_K", "T_C"]
        for oxide in OXIDES.keys():
            fieldnames.extend([f"GM_{oxide}", f"GM_system_{oxide}", f"G_metal_{oxide}",
                               f"dG_{oxide}_per_O2", f"phases_{oxide}", f"oxide_phase_{oxide}"])

        with open(OUTPUT_FILE, 'w', newline='') as f:
            writer = csv.DictWriter(f, fieldnames=fieldnames)
            writer.writeheader()
            for T in temperatures:
                row = {k: results[T].get(k, "") for k in fieldnames}
                writer.writerow(row)

        print(f"Done! {len(temperatures)} rows written.")
        print("=" * 70)


if __name__ == "__main__":
    main()
```

D.2 extract_ternary_reactions.py

Runs on: OSU ETS VM (requires TC-Python + TCOX14 [4])
Runtime: ∼10 minutes
Output: `data/tcpython/raw/ternary_reaction_energies.csv` (414 rows)
Used in: Section 4 (Ternary Reaction Modeling), Table 1

The most important script in the project. For each of 15 Cu-M-O ternary systems, it sets the composition to the known ternary compound stoichiometry (e.g., $x_{\mathrm{Cu}} = 1/7$, $x_{\mathrm{Fe}} = 2/7$, $x_{\mathrm{O}} = 4/7$ for $CuFe_2O_4$) and computes the system Gibbs energy via TC-Python equilibrium. The reaction ΔG is the difference between product and reactant system energies. A negative value means the oxide captures copper.

```
#!/usr/bin/env python3
"""
Extract Gibbs energies for TERNARY REACTION products (Cu + oxide → complex).

This script calculates the Gibbs energy of ternary compound formation
for copper interacting with each screening oxide. The key question:
  "Can Cu react with MOx to form a ternary compound (spinel, vanadate, etc.)?"

Method for each oxide (e.g., Al2O3):
  1. Set up Cu-M-O ternary system in TCOX14
  2. At the ternary compound stoichiometry (e.g., CuAl2O4), calculate equilibrium
  3. Extract G of the ternary phase (SPINEL, etc.) if stable
  4. Also extract G of the binary oxide phase and Cu metal for comparison
  5. Calculate: dG_rxn = G(CuMOy) - G(Cu) - G(MOx)
  6. If dG_rxn < 0, the reaction is favorable → oxide can capture copper

Approach:
  For each oxide at each temperature, we run TWO equilibrium calculations:
    A) "Reactant side": separate Cu metal + pure oxide
       (reuse G_metal and G_oxide from extract_oxide_gibbs.py data)
    B) "Product side": Cu-M-O system at ternary compound stoichiometry
       → let TC find the equilibrium phases (may include spinel, delafossite, etc.)
       → use SYSTEM Gibbs energy (GM * total atoms) as G_products

  dG_rxn = G_products_system - G_reactants_sum

  If the system prefers to form a ternary compound, G_products < G_reactants
  and dG_rxn < 0 (favorable).

Output: ../../data/tcpython/raw/ternary_reaction_energies.csv

Run on OSU lab machine:
  "C:\\Program Files\\Thermo-Calc\\2025b\\python\\python.exe" extract_ternary_reactions.py
"""

import csv
from pathlib import Path
from datetime import datetime

from tc_python import *

# ============================================================================
# Configuration
# ============================================================================
T_MIN = 800
T_MAX = 1900
T_STEP = 50  # finer step in steelmaking range

DATABASE = "TCOX14"

SCRIPT_DIR = Path(__file__).parent
OUTPUT_DIR = SCRIPT_DIR.parent.parent / "data" / "tcpython" / "raw"
OUTPUT_FILE = OUTPUT_DIR / "ternary_reaction_energies.csv"

# ============================================================================
```

```
# Ternary compound definitions
#
# For each oxide, we define:
#   - elements: [METAL, "O"] for the binary oxide (Cu is always added)
#   - ternary_formulas: list of known Cu-M-O compounds to check
#   - For each ternary formula:
#       - name: human-readable name
#       - composition: {element: mole_fraction} for the ternary stoichiometry
#       - atoms_per_formula: total atoms in one formula unit
#       - reaction: string description of the balanced reaction
#       - phase_hints: TC phase names to look for (SPINEL, etc.)
#       - reactant_oxide_atoms: atoms per formula in the reactant oxide
#       - cu_atoms: Cu atoms consumed per formula unit of product
#
# Stoichiometric compositions for ternary compounds:
#   CuAl2O4: Cu=1, Al=2, O=4 → total 7 atoms → X_Cu=1/7, X_Al=2/7, X_O=4/7
#   CuFe2O4: Cu=1, Fe=2, O=4 → total 7 atoms → same fractions
#   CuCr2O4: Cu=1, Cr=2, O=4 → total 7 atoms → same fractions
#   CuMn2O4: Cu=1, Mn=2, O=4 → total 7 atoms → same fractions
#   CuV2O6:  Cu=1, V=2, O=6  → total 9 atoms → X_Cu=1/9, X_V=2/9, X_O=6/9
#   CuTiO3:  Cu=1, Ti=1, O=3 → total 5 atoms → X_Cu=1/5, X_Ti=1/5, X_O=3/5
#   CuSiO3:  Cu=1, Si=1, O=3 → total 5 atoms → same as CuTiO3
#   CuAlO2:  Cu=1, Al=1, O=2 → total 4 atoms → X_Cu=1/4, X_Al=1/4, X_O=2/4
# ==============================================================================

TERNARY_SYSTEMS = [
    # --- SPINEL-FORMING OXIDES ---
    # Spinel: CuM2O4 = Cu + M2O3 + 0.5 O2
    #   Reactant stoich per formula: 1 Cu + 1 M2O3 (5 atoms) + 0.5 O2 (1 O atom)
    #   So: n_Cu=1, n_oxide=1, n_O2=0.5, oxide_atoms=5
    {
        "oxide": "Al2O3",
        "metal": "AL",
        "ternaries": [
            {
                "product": "CuAl2O4",
                "name": "copper aluminate spinel",
                "reaction": "Cu + Al2O3 + 0.5 O2 -> CuAl2O4",
                "X_Cu": 1/7, "X_M": 2/7, "X_O": 4/7,
                "atoms_per_formula": 7,
                "phase_hints": ["SPINEL"],
                # Reactant-side stoichiometry for dG_rxn
                "n_Cu": 1,          # moles Cu metal consumed
                "n_oxide_fu": 1,    # moles of oxide formula units consumed
                "oxide_atoms": 5,   # atoms per oxide formula unit (Al2O3)
                "n_O2": 0.5,        # moles O2 consumed (can be 0 if no extra O)
            },
            {
                "product": "CuAlO2",
                "name": "delafossite",
                "reaction": "Cu + 0.5 Al2O3 + 0.25 O2 -> CuAlO2",
                "X_Cu": 1/4, "X_M": 1/4, "X_O": 2/4,
                "atoms_per_formula": 4,
                "phase_hints": ["DELAFOSSITE", "CUALUMINATE", "CUALOXID"],
                "n_Cu": 1, "n_oxide_fu": 0.5, "oxide_atoms": 5, "n_O2": 0.25,
            },
        ],
    },
    {
        "oxide": "Cr2O3",
        "metal": "CR",
        "ternaries": [
            {
                "product": "CuCr2O4",
                "name": "copper chromite spinel",
                "reaction": "Cu + Cr2O3 + 0.5 O2 -> CuCr2O4",
                "X_Cu": 1/7, "X_M": 2/7, "X_O": 4/7,
                "atoms_per_formula": 7,
                "phase_hints": ["SPINEL"],
                "n_Cu": 1, "n_oxide_fu": 1, "oxide_atoms": 5, "n_O2": 0.5,
```

```
            },
        ],
    },
    {
        "oxide": "MnO",
        "metal": "MN",
        "ternaries": [
            {
                "product": "CuMn2O4",
                "name": "copper manganite spinel",
                "reaction": "Cu + 2MnO + O2 -> CuMn2O4",
                "X_Cu": 1/7, "X_M": 2/7, "X_O": 4/7,
                "atoms_per_formula": 7,
                "phase_hints": ["SPINEL"],
                "n_Cu": 1, "n_oxide_fu": 2, "oxide_atoms": 2, "n_O2": 1.0,
            },
        ],
    },
    {
        "oxide": "FeO",
        "metal": "FE",
        "ternaries": [
            {
                "product": "CuFe2O4",
                "name": "copper ferrite spinel",
                "reaction": "Cu + 2FeO + O2 -> CuFe2O4",
                "X_Cu": 1/7, "X_M": 2/7, "X_O": 4/7,
                "atoms_per_formula": 7,
                "phase_hints": ["SPINEL"],
                "n_Cu": 1, "n_oxide_fu": 2, "oxide_atoms": 2, "n_O2": 1.0,
            },
        ],
    },
    # --- VANADATE/COMPLEX OXIDE ---
    {
        "oxide": "V2O5",
        "metal": "V",
        "ternaries": [
            {
                "product": "CuV2O6",
                "name": "copper vanadate",
                "reaction": "Cu + V2O5 + 0.5 O2 -> CuV2O6",
                "X_Cu": 1/9, "X_M": 2/9, "X_O": 6/9,
                "atoms_per_formula": 9,
                "phase_hints": ["VANADATE", "CUV2O6"],
                "n_Cu": 1, "n_oxide_fu": 1, "oxide_atoms": 7, "n_O2": 0.5,
            },
            {
                "product": "Cu3V2O8",
                "name": "copper orthovanadate",
                "reaction": "3Cu + V2O5 + 1.5 O2 -> Cu3V2O8",
                "X_Cu": 3/13, "X_M": 2/13, "X_O": 8/13,
                "atoms_per_formula": 13,
                "phase_hints": ["ORTHOVANADATE", "CU3V2O8"],
                "n_Cu": 3, "n_oxide_fu": 1, "oxide_atoms": 7, "n_O2": 1.5,
            },
        ],
    },
    # --- TITANATE ---
    {
        "oxide": "TiO2",
        "metal": "TI",
        "ternaries": [
            {
                "product": "CuTiO3",
                "name": "copper titanate",
                "reaction": "Cu + TiO2 + 0.5 O2 -> CuTiO3",
                "X_Cu": 1/5, "X_M": 1/5, "X_O": 3/5,
                "atoms_per_formula": 5,
                "phase_hints": ["ILMENITE", "CUTIO3", "PEROVSKITE"],
```

```
                "n_Cu": 1, "n_oxide_fu": 1, "oxide_atoms": 3, "n_O2": 0.5,
            },
        ],
    },
    # --- SILICATE ---
    {
        "oxide": "SiO2",
        "metal": "SI",
        "ternaries": [
            {
                "product": "CuSiO3",
                "name": "copper metasilicate",
                "reaction": "Cu + SiO2 + 0.5 O2 -> CuSiO3",
                "X_Cu": 1/5, "X_M": 1/5, "X_O": 3/5,
                "atoms_per_formula": 5,
                "phase_hints": ["PYROXENE", "CUSIO3", "WOLLASTONITE"],
                "n_Cu": 1, "n_oxide_fu": 1, "oxide_atoms": 3, "n_O2": 0.5,
            },
            {
                "product": "Cu2SiO4",
                "name": "copper orthosilicate",
                "reaction": "2Cu + SiO2 + O2 -> Cu2SiO4",
                "X_Cu": 2/7, "X_M": 1/7, "X_O": 4/7,
                "atoms_per_formula": 7,
                "phase_hints": ["OLIVINE", "CU2SIO4"],
                "n_Cu": 2, "n_oxide_fu": 1, "oxide_atoms": 3, "n_O2": 1.0,
            },
        ],
    },
    # --- SIMPLE OXIDES (less likely to form ternaries, but check) ---
    {
        "oxide": "MgO",
        "metal": "MG",
        "ternaries": [
            {
                "product": "CuMgO2",
                "name": "copper magnesioxide (hypothetical)",
                "reaction": "Cu + MgO + 0.5 O2 -> CuMgO2",
                "X_Cu": 1/4, "X_M": 1/4, "X_O": 2/4,
                "atoms_per_formula": 4,
                "phase_hints": ["DELAFOSSITE", "CUMGO2"],
                "n_Cu": 1, "n_oxide_fu": 1, "oxide_atoms": 2, "n_O2": 0.5,
            },
        ],
    },
    {
        "oxide": "CaO",
        "metal": "CA",
        "ternaries": [
            {
                "product": "CuCaO2",
                "name": "copper calcioxide (hypothetical)",
                "reaction": "Cu + CaO + 0.5 O2 -> CuCaO2",
                "X_Cu": 1/4, "X_M": 1/4, "X_O": 2/4,
                "atoms_per_formula": 4,
                "phase_hints": ["DELAFOSSITE", "CUCAO2"],
                "n_Cu": 1, "n_oxide_fu": 1, "oxide_atoms": 2, "n_O2": 0.5,
            },
        ],
    },
    {
        "oxide": "ZrO2",
        "metal": "ZR",
        "ternaries": [
            {
                "product": "CuZrO3",
                "name": "copper zirconate (hypothetical)",
                "reaction": "Cu + ZrO2 + 0.5 O2 -> CuZrO3",
                "X_Cu": 1/5, "X_M": 1/5, "X_O": 3/5,
                "atoms_per_formula": 5,
```

```
                "phase_hints": ["PEROVSKITE", "CUZRO3"],
                "n_Cu": 1, "n_oxide_fu": 1, "oxide_atoms": 3, "n_O2": 0.5,
            },
        ],
    },
    # --- MOONSHOT OXIDES ---
    {
        "oxide": "NiO",
        "metal": "NI",
        "ternaries": [
            {
                "product": "CuNiO2",
                "name": "copper nickelate",
                "reaction": "Cu + NiO + 0.5 O2 -> CuNiO2",
                "X_Cu": 1/4, "X_M": 1/4, "X_O": 2/4,
                "atoms_per_formula": 4,
                "phase_hints": ["DELAFOSSITE", "CUNIO2"],
                "n_Cu": 1, "n_oxide_fu": 1, "oxide_atoms": 2, "n_O2": 0.5,
            },
        ],
    },
    {
        "oxide": "CoO",
        "metal": "CO",
        "ternaries": [
            {
                "product": "CuCo2O4",
                "name": "copper cobaltite spinel",
                "reaction": "Cu + 2CoO + O2 -> CuCo2O4",
                "X_Cu": 1/7, "X_M": 2/7, "X_O": 4/7,
                "atoms_per_formula": 7,
                "phase_hints": ["SPINEL"],
                "n_Cu": 1, "n_oxide_fu": 2, "oxide_atoms": 2, "n_O2": 1.0,
            },
        ],
    },
    {
        "oxide": "La2O3",
        "metal": "LA",
        "ternaries": [
            {
                "product": "CuLaO2",
                "name": "copper lanthanum oxide",
                "reaction": "Cu + 0.5 La2O3 + 0.25 O2 -> CuLaO2",
                "X_Cu": 1/4, "X_M": 1/4, "X_O": 2/4,
                "atoms_per_formula": 4,
                "phase_hints": ["DELAFOSSITE", "CULAO2"],
                "n_Cu": 1, "n_oxide_fu": 0.5, "oxide_atoms": 5, "n_O2": 0.25,
            },
        ],
    },
    {
        "oxide": "CeO2",
        "metal": "CE",
        "ternaries": [
            {
                "product": "CuCeO3",
                "name": "copper cerate (hypothetical)",
                "reaction": "Cu + CeO2 + 0.5 O2 -> CuCeO3",
                "X_Cu": 1/5, "X_M": 1/5, "X_O": 3/5,
                "atoms_per_formula": 5,
                "phase_hints": ["PEROVSKITE", "CUCEO3"],
                "n_Cu": 1, "n_oxide_fu": 1, "oxide_atoms": 3, "n_O2": 0.5,
            },
        ],
    },
    # B2O3 requires special handling (B may not coexist with Cu in TCOX14)
    {
        "oxide": "B2O3",
        "metal": "B",
```

```
        "ternaries": [
            {
                "product": "CuB2O4",
                "name": "copper borate",
                "reaction": "Cu + B2O3 + 0.5 O2 -> CuB2O4",
                "X_Cu": 1/7, "X_M": 2/7, "X_O": 4/7,
                "atoms_per_formula": 7,
                "phase_hints": ["BORATE", "CUBO", "CUB2O4"],
                "n_Cu": 1, "n_oxide_fu": 1, "oxide_atoms": 5, "n_O2": 0.5,
            },
        ],
    },
]

# PbO note: PB is not in TCOX14 (would need SSUB3 fallback).
# We skip PbO ternaries since Pb is toxic and not in TCOX14.


def find_phase_gm(result, phase_hints):
    """Search for a specific phase and return its GM value."""
    stable = result.get_stable_phases()
    for hint in phase_hints:
        for phase in stable:
            if hint.upper() in phase.upper():
                try:
                    gm = result.get_value_of(f"GM({phase})")
                    return gm, phase
                except Exception:
                    pass
    return None, None


def main():
    print("=" * 70)
    print("TC-Python: Ternary Reaction Energies (Cu + oxide -> complex)")
    print("=" * 70)
    print(f"Database: {DATABASE}")
    print(f"Temperature range: {T_MIN}-{T_MAX} K")
    print(f"Output: {OUTPUT_FILE}")
    print(f"Started: {datetime.now().isoformat()}")
    print()

    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    temperatures = list(range(T_MIN, T_MAX + 1, T_STEP))

    # CSV rows
    all_rows = []

    with TCPython() as session:
        print("Connected to Thermo-Calc\n")

        # ==================================================================
        # Step 1: Get Cu metal reference energy at each temperature
        # ==================================================================
        print("--- Getting Cu metal reference ---")
        G_Cu_metal = {}
        try:
            cu_system = (session
                         .select_database_and_elements(DATABASE, ["CU", "O"])
                         .get_system())
            for T in temperatures:
                calc = cu_system.with_single_equilibrium_calculation()
                calc.set_condition(ThermodynamicQuantity.temperature(), T)
                calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
                # Nearly pure Cu (tiny O to keep system defined)
                calc.set_condition(
                    ThermodynamicQuantity.mole_fraction_of_a_component("O"),
                    0.0001
                )
                result = calc.calculate()
```

```
                G_Cu_metal[T] = result.get_value_of("GM")
            print(f"  Cu(metal) at 1500K: {G_Cu_metal.get(1500, 'N/A'):.1f} J/mol-atoms\n")
        except Exception as e:
            print(f"  ERROR getting Cu reference: {e}\n")
            return

        # ====================================================================
        # Step 2: Get O2 gas reference (per mol O2)
        # ====================================================================
        print("--- Getting O2 gas reference ---")
        G_O2 = {}
        try:
            o2_system = (session
                         .select_database_and_elements(DATABASE, ["O"])
                         .get_system())
            for T in temperatures:
                calc = o2_system.with_single_equilibrium_calculation()
                calc.set_condition(ThermodynamicQuantity.temperature(), T)
                calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
                result = calc.calculate()
                G_O2[T] = 2 * result.get_value_of("GM")  # per mol O2
            print(f"  O2(gas) at 1500K: {G_O2.get(1500, 'N/A'):.1f} J/mol-O2\n")
        except Exception as e:
            print(f"  ERROR getting O2 reference: {e}\n")
            return

        # ====================================================================
        # Step 3: For each ternary system, calculate product-side equilibrium
        # ====================================================================
        for sys_def in TERNARY_SYSTEMS:
            oxide_name = sys_def["oxide"]
            metal_el = sys_def["metal"]
            elements = ["CU", metal_el, "O"]

            print(f"\n{'='*60}")
            print(f"System: Cu-{metal_el}-O  (oxide = {oxide_name})")
            print(f"{'='*60}")

            # Try to set up this ternary system
            try:
                ternary_system = (session
                                  .select_database_and_elements(DATABASE, elements)
                                  .get_system())
            except Exception as e:
                print(f"  SYSTEM SETUP ERROR: {e}")
                print(f"  (Element {metal_el} may not be in {DATABASE})")
                # Write error rows
                for tern in sys_def["ternaries"]:
                    for T in temperatures:
                        all_rows.append({
                            "T_K": T,
                            "T_C": T - 273.15,
                            "oxide": oxide_name,
                            "product": tern["product"],
                            "product_name": tern["name"],
                            "reaction": tern["reaction"],
                            "GM_system_product": "",
                            "stable_phases": f"ERROR: {e}",
                            "ternary_phase_found": "",
                            "GM_ternary_phase": "",
                            "G_Cu_metal": "",
                            "G_O2": "",
                            "dG_rxn_system_J": "",
                            "dG_rxn_system_kJ": "",
                            "notes": f"Element {metal_el} not in {DATABASE}",
                        })
                continue

            # --- Also get the BINARY oxide reference in this ternary system ---
            # Calculate at the binary oxide stoichiometry (no Cu)
```

```
            print(f"  Getting {oxide_name} reference in ternary system...")
            G_binary_oxide = {}
            oxide_X_O_map = {
                "Al2O3": 3/5, "Cr2O3": 3/5, "La2O3": 3/5,
                "MnO": 1/2, "FeO": 1/2, "MgO": 1/2, "CaO": 1/2,
                "NiO": 1/2, "CoO": 1/2,
                "V2O5": 5/7, "TiO2": 2/3, "SiO2": 2/3, "ZrO2": 2/3,
                "CeO2": 2/3, "B2O3": 3/5,
            }
            oxide_atoms_map = {
                "Al2O3": 5, "Cr2O3": 5, "La2O3": 5,
                "MnO": 2, "FeO": 2, "MgO": 2, "CaO": 2,
                "NiO": 2, "CoO": 2,
                "V2O5": 7, "TiO2": 3, "SiO2": 3, "ZrO2": 3,
                "CeO2": 3, "B2O3": 5,
            }
            X_O_oxide = oxide_X_O_map.get(oxide_name, 0.5)
            atoms_oxide = oxide_atoms_map.get(oxide_name, 2)

            for T in temperatures:
                try:
                    calc = ternary_system.with_single_equilibrium_calculation()
                    calc.set_condition(ThermodynamicQuantity.temperature(), T)
                    calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
                    # Set to binary oxide composition (X_Cu ~ 0)
                    calc.set_condition(
                        ThermodynamicQuantity.mole_fraction_of_a_component("CU"),
                        0.0001
                    )
                    calc.set_condition(
                        ThermodynamicQuantity.mole_fraction_of_a_component("O"),
                        X_O_oxide
                    )
                    result = calc.calculate()
                    G_binary_oxide[T] = result.get_value_of("GM")
                except Exception as e:
                    G_binary_oxide[T] = None

            if G_binary_oxide.get(1500):
                print(f"  {oxide_name} GM at 1500K: {G_binary_oxide[1500]:.1f} J/mol-atoms")

            # --- Now calculate at each TERNARY compound stoichiometry ---
            for tern in sys_def["ternaries"]:
                product = tern["product"]
                X_Cu = tern["X_Cu"]
                X_M = tern["X_M"]
                X_O = tern["X_O"]
                atoms = tern["atoms_per_formula"]
                phase_hints = tern["phase_hints"]

                print(f"\n  --- {product} ({tern['name']}) ---")
                print(f"  Composition: X_Cu={X_Cu:.4f}, X_{metal_el}={X_M:.4f}, X_O={X_O:.4f}")
                print(f"  Reaction: {tern['reaction']}")

                success = 0
                for T in temperatures:
                    row = {
                        "T_K": T,
                        "T_C": T - 273.15,
                        "oxide": oxide_name,
                        "product": product,
                        "product_name": tern["name"],
                        "reaction": tern["reaction"],
                    }

                    try:
                        calc = ternary_system.with_single_equilibrium_calculation()
                        calc.set_condition(ThermodynamicQuantity.temperature(), T)
                        calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
                        calc.set_condition(
```

```
                        ThermodynamicQuantity.mole_fraction_of_a_component("CU"),
                        X_Cu
                    )
                    calc.set_condition(
                        ThermodynamicQuantity.mole_fraction_of_a_component("O"),
                        X_O
                    )
                    result = calc.calculate()

                    stable = result.get_stable_phases()
                    GM_system = result.get_value_of("GM")

                    # Look for ternary phase
                    gm_ternary, ternary_phase = find_phase_gm(result, phase_hints)

                    row["GM_system_product"] = GM_system
                    row["stable_phases"] = "; ".join(stable)
                    row["ternary_phase_found"] = ternary_phase if ternary_phase else ""
                    row["GM_ternary_phase"] = gm_ternary if gm_ternary else ""
                    row["G_Cu_metal"] = G_Cu_metal[T]
                    row["G_O2"] = G_O2[T]

                    # ------------------------------------------------
                    # Reaction dG calculation
                    #
                    # For the balanced reaction (e.g.):
                    #   Cu + Al2O3 + 0.5 O2 -> CuAl2O4
                    #
                    # G_products = GM_system * atoms_per_formula
                    #   (total G of 1 formula unit at this composition)
                    #
                    # G_reactants = n_Cu * G(Cu per atom)
                    #             + n_oxide_fu * oxide_atoms * G(oxide per atom)
                    #             + n_O2 * G(O2 per mol)
                    #
                    # dG_rxn = G_products - G_reactants
                    # If dG_rxn < 0, ternary compound formation
                    # is thermodynamically favorable.
                    # ------------------------------------------------

                    G_oxide_ref = G_binary_oxide.get(T)
                    if G_oxide_ref is not None:
                        n_Cu = tern["n_Cu"]
                        n_oxide_fu = tern["n_oxide_fu"]
                        ox_atoms = tern["oxide_atoms"]
                        n_O2 = tern["n_O2"]

                        # Product: system GM * total atoms in formula
                        G_products = GM_system * atoms

                        # Reactants: Cu metal + oxide + O2 gas
                        # G_Cu_metal is GM per atom of Cu
                        # G_oxide_ref is GM per atom of oxide
                        # G_O2 is already per mol O2
                        G_reactants = (n_Cu * G_Cu_metal[T]
                                       + n_oxide_fu * ox_atoms * G_oxide_ref
                                       + n_O2 * G_O2[T])

                        dG_rxn_J = G_products - G_reactants
                        dG_rxn_kJ = dG_rxn_J / 1000

                        row["dG_rxn_system_J"] = dG_rxn_J
                        row["dG_rxn_system_kJ"] = dG_rxn_kJ
                        row["notes"] = ""
                    else:
                        row["dG_rxn_system_J"] = ""
                        row["dG_rxn_system_kJ"] = ""
                        row["notes"] = "No binary oxide reference"

                    success += 1
```

```python

                    except Exception as e:
                        row["GM_system_product"] = ""
                        row["stable_phases"] = f"ERROR: {e}"
                        row["ternary_phase_found"] = ""
                        row["GM_ternary_phase"] = ""
                        row["G_Cu_metal"] = G_Cu_metal.get(T, "")
                        row["G_O2"] = G_O2.get(T, "")
                        row["dG_rxn_system_J"] = ""
                        row["dG_rxn_system_kJ"] = ""
                        row["notes"] = str(e)

                    all_rows.append(row)

                print(f"  Completed: {success}/{len(temperatures)} temperatures")

                # Print sample result at steelmaking temp
                sample_T = 1800
                sample_rows = [r for r in all_rows
                               if r["T_K"] == sample_T
                               and r["product"] == product
                               and r.get("dG_rxn_system_kJ")]
                if sample_rows:
                    sr = sample_rows[0]
                    dG = sr["dG_rxn_system_kJ"]
                    phases = sr["stable_phases"]
                    tern_found = sr["ternary_phase_found"]
                    verdict = "FAVORABLE (Cu captured)" if dG < 0 else "UNFAVORABLE"
                    print(f"  At 1800K: dG_rxn = {dG:+.1f} kJ/mol → {verdict}")
                    print(f"  Phases: {phases}")
                    if tern_found:
                        print(f"  Ternary phase: {tern_found}")

    # ======================================================================
    # Write CSV
    # ======================================================================
    fieldnames = [
        "T_K", "T_C", "oxide", "product", "product_name", "reaction",
        "GM_system_product", "stable_phases", "ternary_phase_found",
        "GM_ternary_phase", "G_Cu_metal", "G_O2",
        "dG_rxn_system_J", "dG_rxn_system_kJ", "notes",
    ]

    with open(OUTPUT_FILE, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(all_rows)

    print(f"\n{'='*70}")
    print(f"CSV written to: {OUTPUT_FILE}")
    print(f"{len(all_rows)} rows ({len(TERNARY_SYSTEMS)} oxide systems)")
    print(f"Finished: {datetime.now().isoformat()}")
    print("=" * 70)

    # Summary table
    print("\n" + "=" * 70)
    print("SUMMARY: Ternary Reaction Feasibility at 1527°C (1800 K)")
    print("=" * 70)
    print(f"{'Product':<16} {'dG_rxn (kJ)':<14} {'Ternary Phase?':<20} {'Verdict':<16}")
    print("-" * 70)

    for sys_def in TERNARY_SYSTEMS:
        for tern in sys_def["ternaries"]:
            product = tern["product"]
            matches = [r for r in all_rows
                       if r["T_K"] == 1800 and r["product"] == product]
            if matches and matches[0].get("dG_rxn_system_kJ"):
                dG = matches[0]["dG_rxn_system_kJ"]
                phase = matches[0].get("ternary_phase_found", "")
                verdict = "FAVORABLE" if dG < 0 else "Unfavorable"
```

```
                print(f"{product:<16} {dG:>+10.1f}    {phase or 'not found':<20} {verdict:<16}")
            else:
                notes = matches[0].get("notes", "no data") if matches else "no data"
                print(f"{product:<16} {'N/A':>10}    {'--':<20} {notes:<16}")

    print("=" * 70)


if __name__ == "__main__":
    main()
```

D.3 cufe2o4_alternative_reaction.py

Runs on: OSU ETS VM (requires TC-Python + TCOX14 [4])
Runtime: ∼3 minutes
Output: `data/tcpython/raw/cufe2o4_alternative_reaction.csv`
Used in: Section 10 ($CuFe_2O_4$ Deep Dive)

$CuFe_2O_4$ has the strongest driving force (−112 kJ), but how much of that comes from Cu capture vs. $Fe^{2+} \rightarrow Fe^{3+}$ oxidation? This script computes the alternative reaction using Fe_2O_3 (Fe already at +3) instead of FeO. The difference isolates the Fe-oxidation bonus. Result: Cu capture alone contributes −46 kJ (41%), confirming the reaction is not just riding on iron oxidation.

```python
#!/usr/bin/env python3
"""
Isolate Cu-capture energy from Fe-oxidation energy for CuFe2O4.

The original ternary reaction uses FeO (Fe at +2):
    Cu + 2FeO + O2 -> CuFe2O4       (dG ~ -112 kJ at 1800K)

This includes BOTH Cu capture AND Fe2+ -> Fe3+ oxidation energy.
To isolate the Cu-capture contribution, use Fe2O3 (Fe already at +3):
    Cu + Fe2O3 + 0.5 O2 -> CuFe2O4

The difference between original and alternative dG = Fe oxidation energy.
If the alternative dG is still strongly negative, Cu capture alone drives
the reaction (the desired mechanism). If it's near zero, most of the
-112 kJ came from Fe oxidation (less useful).

Expected: alternative dG less negative than -112 kJ, closer to CuAl2O4's -32 kJ.

Output: ../../data/tcpython/raw/cufe2o4_alternative_reaction.csv

Run on OSU lab machine:
  "C:\\Program Files\\Thermo-Calc\\2025b\\python\\python.exe" cufe2o4_alternative_reaction.py
"""

import csv
from pathlib import Path
from datetime import datetime

from tc_python import *

# ==============================================================================
# Configuration
# ==============================================================================
T_MIN = 800
T_MAX = 1900
T_STEP = 50  # 23 temperature points

DATABASE = "TCOX14"

SCRIPT_DIR = Path(__file__).parent
OUTPUT_DIR = SCRIPT_DIR.parent.parent / "data" / "tcpython" / "raw"
OUTPUT_FILE = OUTPUT_DIR / "cufe2o4_alternative_reaction.csv"

# Product: CuFe2O4 (spinel)
# Cu=1, Fe=2, O=4 -> 7 atoms total
PRODUCT_X_CU = 1/7
PRODUCT_X_O = 4/7
PRODUCT_ATOMS = 7

# Alternative reactant: Fe2O3 (hematite)
# Fe=2, O=3 -> 5 atoms total
FE2O3_X_FE = 2/5  # = 0.4
FE2O3_X_O = 3/5   # = 0.6
FE2O3_ATOMS = 5

# Original reactant: FeO (wustite)
# Fe=1, O=1 -> 2 atoms total
```

```python
FEO_X_O = 1/2
FEO_ATOMS = 2

# Reaction stoichiometry:
#   Alternative: Cu + Fe2O3 + 0.5 O2 -> CuFe2O4
#     n_Cu=1, n_Fe2O3=1 (5 atoms), n_O2=0.5
#   Original:   Cu + 2FeO + O2 -> CuFe2O4
#     n_Cu=1, n_FeO=2 (2 atoms each), n_O2=1.0


def find_phase_gm(result, phase_hints):
    """Search for a specific phase and return its GM value."""
    stable = result.get_stable_phases()
    for hint in phase_hints:
        for phase in stable:
            if hint.upper() in phase.upper():
                try:
                    gm = result.get_value_of("GM({})".format(phase))
                    return gm, phase
                except Exception:
                    pass
    return None, None


def main():
    print("=" * 70)
    print("TC-Python: CuFe2O4 Alternative Reaction (Fe2O3 vs FeO)")
    print("=" * 70)
    print("Database: {}".format(DATABASE))
    print("Temperature range: {}-{} K, step {} K".format(T_MIN, T_MAX, T_STEP))
    print("Output: {}".format(OUTPUT_FILE))
    print("Started: {}".format(datetime.now().isoformat()))
    print()
    print("Alternative: Cu + Fe2O3 + 0.5 O2 -> CuFe2O4  (Fe already +3)")
    print("Original:    Cu + 2FeO  + O2     -> CuFe2O4  (Fe at +2)")
    print()

    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    temperatures = list(range(T_MIN, T_MAX + 1, T_STEP))

    all_rows = []

    with TCPython() as session:
        print("Connected to Thermo-Calc\n")

        # ================================================================
        # Step 1: Cu metal reference (Cu-O system, X_O ~ 0)
        # ================================================================
        print("--- Getting Cu metal reference ---")
        G_Cu_metal = {}
        try:
            cu_system = (session
                         .select_database_and_elements(DATABASE, ["CU", "O"])
                         .get_system())
            for T in temperatures:
                calc = cu_system.with_single_equilibrium_calculation()
                calc.set_condition(ThermodynamicQuantity.temperature(), T)
                calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
                calc.set_condition(
                    ThermodynamicQuantity.mole_fraction_of_a_component("O"),
                    0.0001
                )
                result = calc.calculate()
                G_Cu_metal[T] = result.get_value_of("GM")
            print("  Cu(metal) at 1500K: {:.1f} J/mol-atoms\n".format(
                G_Cu_metal.get(1500, 0)))
        except Exception as e:
            print("  ERROR getting Cu reference: {}\n".format(e))
            return

```

```
        # ================================================================
        # Step 2: O2 gas reference (per mol O2)
        # ================================================================
        print("--- Getting O2 gas reference ---")
        G_O2 = {}
        try:
            o2_system = (session
                         .select_database_and_elements(DATABASE, ["O"])
                         .get_system())
            for T in temperatures:
                calc = o2_system.with_single_equilibrium_calculation()
                calc.set_condition(ThermodynamicQuantity.temperature(), T)
                calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
                result = calc.calculate()
                G_O2[T] = 2 * result.get_value_of("GM")  # per mol O2
            print("  O2(gas) at 1500K: {:.1f} J/mol-O2\n".format(
                G_O2.get(1500, 0)))
        except Exception as e:
            print("  ERROR getting O2 reference: {}\n".format(e))
            return

        # ================================================================
        # Step 3: Set up Cu-Fe-O ternary system
        # ================================================================
        print("--- Setting up Cu-Fe-O ternary system ---")
        try:
            ternary_system = (session
                              .select_database_and_elements(DATABASE, ["CU", "FE", "O"])
                              .get_system())
        except Exception as e:
            print("  SYSTEM SETUP ERROR: {}".format(e))
            return

        # ================================================================
        # Step 4: Fe2O3 reference (alternative reactant)
        # ================================================================
        print("--- Getting Fe2O3 reference (X_Fe=0.4, X_O=0.6) ---")
        G_Fe2O3 = {}
        for T in temperatures:
            try:
                calc = ternary_system.with_single_equilibrium_calculation()
                calc.set_condition(ThermodynamicQuantity.temperature(), T)
                calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
                calc.set_condition(
                    ThermodynamicQuantity.mole_fraction_of_a_component("CU"),
                    0.0001
                )
                calc.set_condition(
                    ThermodynamicQuantity.mole_fraction_of_a_component("O"),
                    FE2O3_X_O
                )
                result = calc.calculate()
                G_Fe2O3[T] = result.get_value_of("GM")
            except Exception as e:
                G_Fe2O3[T] = None
                print("  Fe2O3 error at {}K: {}".format(T, e))

        if G_Fe2O3.get(1500):
            print("  Fe2O3 GM at 1500K: {:.1f} J/mol-atoms".format(G_Fe2O3[1500]))

        # ================================================================
        # Step 5: FeO reference (original reactant)
        # ================================================================
        print("--- Getting FeO reference (X_O=0.5) ---")
        G_FeO = {}
        for T in temperatures:
            try:
                calc = ternary_system.with_single_equilibrium_calculation()
                calc.set_condition(ThermodynamicQuantity.temperature(), T)
                calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
```

```
                calc.set_condition(
                    ThermodynamicQuantity.mole_fraction_of_a_component("CU"),
                    0.0001
                )
                calc.set_condition(
                    ThermodynamicQuantity.mole_fraction_of_a_component("O"),
                    FEO_X_O
                )
                result = calc.calculate()
                G_FeO[T] = result.get_value_of("GM")
            except Exception as e:
                G_FeO[T] = None
                print("  FeO error at {}K: {}".format(T, e))

        if G_FeO.get(1500):
            print("  FeO GM at 1500K: {:.1f} J/mol-atoms".format(G_FeO[1500]))

        # ================================================================
        # Step 6: Product-side equilibrium at CuFe2O4 stoichiometry
        # ================================================================
        print("\n--- Calculating CuFe2O4 product at each temperature ---")
        success = 0
        for T in temperatures:
            row = {
                "T_K": T,
                "T_C": T - 273.15,
            }

            try:
                calc = ternary_system.with_single_equilibrium_calculation()
                calc.set_condition(ThermodynamicQuantity.temperature(), T)
                calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
                calc.set_condition(
                    ThermodynamicQuantity.mole_fraction_of_a_component("CU"),
                    PRODUCT_X_CU
                )
                calc.set_condition(
                    ThermodynamicQuantity.mole_fraction_of_a_component("O"),
                    PRODUCT_X_O
                )
                result = calc.calculate()

                stable = result.get_stable_phases()
                GM_system = result.get_value_of("GM")
                gm_spinel, spinel_phase = find_phase_gm(result, ["SPINEL"])

                row["GM_system_product"] = GM_system
                row["stable_phases"] = "; ".join(stable)
                row["spinel_phase_found"] = spinel_phase if spinel_phase else ""
                row["GM_spinel"] = gm_spinel if gm_spinel else ""
                row["G_Cu_metal"] = G_Cu_metal[T]
                row["G_O2"] = G_O2[T]

                # G_products = GM_system * atoms_per_formula
                G_products = GM_system * PRODUCT_ATOMS

                # --- Alternative reaction: Cu + Fe2O3 + 0.5 O2 -> CuFe2O4 ---
                G_Fe2O3_ref = G_Fe2O3.get(T)
                if G_Fe2O3_ref is not None:
                    G_reactants_alt = (1 * G_Cu_metal[T]
                                       + 1 * FE2O3_ATOMS * G_Fe2O3_ref
                                       + 0.5 * G_O2[T])
                    dG_alt_J = G_products - G_reactants_alt
                    dG_alt_kJ = dG_alt_J / 1000
                    row["dG_alternative_J"] = dG_alt_J
                    row["dG_alternative_kJ"] = dG_alt_kJ
                else:
                    row["dG_alternative_J"] = ""
                    row["dG_alternative_kJ"] = ""
```

```
                # --- Original reaction: Cu + 2FeO + O2 -> CuFe2O4 ---
                G_FeO_ref = G_FeO.get(T)
                if G_FeO_ref is not None:
                    G_reactants_orig = (1 * G_Cu_metal[T]
                                        + 2 * FEO_ATOMS * G_FeO_ref
                                        + 1.0 * G_O2[T])
                    dG_orig_J = G_products - G_reactants_orig
                    dG_orig_kJ = dG_orig_J / 1000
                    row["dG_original_J"] = dG_orig_J
                    row["dG_original_kJ"] = dG_orig_kJ
                else:
                    row["dG_original_J"] = ""
                    row["dG_original_kJ"] = ""

                # --- Difference = Fe oxidation contribution ---
                if (row.get("dG_alternative_kJ", "") != ""
                        and row.get("dG_original_kJ", "") != ""):
                    row["dG_Fe_oxidation_kJ"] = dG_orig_kJ - dG_alt_kJ
                else:
                    row["dG_Fe_oxidation_kJ"] = ""

                row["notes"] = ""
                success += 1

            except Exception as e:
                row["GM_system_product"] = ""
                row["stable_phases"] = "ERROR: {}".format(e)
                row["spinel_phase_found"] = ""
                row["GM_spinel"] = ""
                row["G_Cu_metal"] = G_Cu_metal.get(T, "")
                row["G_O2"] = G_O2.get(T, "")
                row["dG_alternative_J"] = ""
                row["dG_alternative_kJ"] = ""
                row["dG_original_J"] = ""
                row["dG_original_kJ"] = ""
                row["dG_Fe_oxidation_kJ"] = ""
                row["notes"] = str(e)

            all_rows.append(row)

        print("  Completed: {}/{} temperatures".format(success, len(temperatures)))

        # Print sample at 1800K
        sample = [r for r in all_rows if r["T_K"] == 1800
                  and r.get("dG_alternative_kJ", "") != ""]
        if sample:
            s = sample[0]
            print("\n  At 1800K:")
            print("    Alternative (Fe2O3): dG = {:+.1f} kJ".format(
                s["dG_alternative_kJ"]))
            print("    Original (FeO):      dG = {:+.1f} kJ".format(
                s["dG_original_kJ"]))
            print("    Fe oxidation contrib: {:.1f} kJ".format(
                s["dG_Fe_oxidation_kJ"]))
            print("    Phases: {}".format(s["stable_phases"]))

    # ====================================================================
    # Write CSV
    # ====================================================================
    fieldnames = [
        "T_K", "T_C", "GM_system_product", "stable_phases",
        "spinel_phase_found", "GM_spinel", "G_Cu_metal", "G_O2",
        "dG_alternative_J", "dG_alternative_kJ",
        "dG_original_J", "dG_original_kJ",
        "dG_Fe_oxidation_kJ", "notes",
    ]

    with open(OUTPUT_FILE, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
```

```
        writer.writerows(all_rows)

    print("\n" + "=" * 70)
    print("CSV written to: {}".format(OUTPUT_FILE))
    print("{} rows".format(len(all_rows)))
    print("Finished: {}".format(datetime.now().isoformat()))
    print("=" * 70)

    # Summary table
    print("\n" + "=" * 70)
    print("SUMMARY: CuFe2O4 Reaction Decomposition")
    print("=" * 70)
    print("{:<8} {:>14} {:>14} {:>14}".format(
        "T (K)", "Alt dG (kJ)", "Orig dG (kJ)", "Fe-ox (kJ)"))
    print("-" * 54)
    for r in all_rows:
        if r.get("dG_alternative_kJ", "") != "":
            print("{:<8} {:>+14.1f} {:>+14.1f} {:>14.1f}".format(
                r["T_K"], r["dG_alternative_kJ"],
                r["dG_original_kJ"], r["dG_Fe_oxidation_kJ"]))
    print("=" * 70)


if __name__ == "__main__":
    main()
```

D.4 cu_activity_vs_oxide.py

Runs on: OSU ETS VM (requires TC-Python + TCOX14 [4])
Runtime: ∼5 minutes
Output: `data/tcpython/raw/cu_activity_vs_oxide.csv`
Used in: Section 7 (Cu Activity and the Gibbs Phase Rule), Fig. 6

Sweeps Cu mole fraction from 0.90 to 0.05 in four Cu-M-O systems at 1800 K, recording a_{Cu} via `AC(CU)`. The result was initially puzzling: activity is flat across the entire sweep. This turned out to be a textbook Gibbs Phase Rule consequence (Section 7), not a bug.

```python
#!/usr/bin/env python3
"""
Measure Cu activity drop as oxide is added to Cu melt at 1800K.

This is the key experimental observable: if an oxide captures Cu, the
activity of Cu in the melt should decrease. A steeper a_Cu drop means
stronger Cu-oxide interaction.

Systems: Cu-Al-O, Cu-Mn-O, Cu-Fe-O, Cu-V-O
Fixed T=1800K, sweep X_Cu from 0.90 to 0.05 in 20 steps.
The non-Cu fraction maintains the oxide's stoichiometric ratio:
  - Al2O3: X_Al:X_O = 2:3
  - MnO:   X_Mn:X_O = 1:1
  - FeO:   X_Fe:X_O = 1:1
  - V2O5:  X_V:X_O  = 2:5

Key API: result.get_value_of("AC(CU)") for Cu activity.

Output: ../../data/tcpython/raw/cu_activity_vs_oxide.csv

Run on OSU lab machine:
  "C:\\Program Files\\Thermo-Calc\\2025b\\python\\python.exe" cu_activity_vs_oxide.py
"""

import csv
from pathlib import Path
from datetime import datetime

from tc_python import *

# ==============================================================================
# Configuration
# ==============================================================================
T_FIXED = 1800  # K (steelmaking temperature)
DATABASE = "TCOX14"

# X_Cu sweep: 0.90 down to 0.05 in 20 steps
N_STEPS = 20
X_CU_VALUES = [0.90 - i * (0.90 - 0.05) / (N_STEPS - 1) for i in range(N_STEPS)]

SCRIPT_DIR = Path(__file__).parent
OUTPUT_DIR = SCRIPT_DIR.parent.parent / "data" / "tcpython" / "raw"
OUTPUT_FILE = OUTPUT_DIR / "cu_activity_vs_oxide.csv"

# Oxide definitions: element, TC symbol, O ratio in non-Cu fraction
# For Cu-M-O at a given X_Cu:
#   X_oxide_total = 1 - X_Cu
#   X_M = X_oxide_total * (metal_atoms / total_oxide_atoms)
#   X_O = X_oxide_total * (oxygen_atoms / total_oxide_atoms)
SYSTEMS = [
    {
        "name": "Cu-Al-O",
        "oxide": "Al2O3",
        "metal": "AL",
        "elements": ["CU", "AL", "O"],
        # Al2O3: 2 Al + 3 O = 5 atoms -> metal_frac = 2/5, O_frac = 3/5
        "metal_frac": 2/5,
        "O_frac": 3/5,
```

```
    },
    {
        "name": "Cu-Mn-O",
        "oxide": "MnO",
        "metal": "MN",
        "elements": ["CU", "MN", "O"],
        # MnO: 1 Mn + 1 O = 2 atoms -> metal_frac = 1/2, O_frac = 1/2
        "metal_frac": 1/2,
        "O_frac": 1/2,
    },
    {
        "name": "Cu-Fe-O",
        "oxide": "FeO",
        "metal": "FE",
        "elements": ["CU", "FE", "O"],
        # FeO: 1 Fe + 1 O = 2 atoms
        "metal_frac": 1/2,
        "O_frac": 1/2,
    },
    {
        "name": "Cu-V-O",
        "oxide": "V2O5",
        "metal": "V",
        "elements": ["CU", "V", "O"],
        # V2O5: 2 V + 5 O = 7 atoms -> metal_frac = 2/7, O_frac = 5/7
        "metal_frac": 2/7,
        "O_frac": 5/7,
    },
]


def main():
    print("=" * 70)
    print("TC-Python: Cu Activity vs Oxide Addition at {}K".format(T_FIXED))
    print("=" * 70)
    print("Database: {}".format(DATABASE))
    print("X_Cu range: {:.2f} to {:.2f} ({} steps)".format(
        X_CU_VALUES[0], X_CU_VALUES[-1], N_STEPS))
    print("Systems: {}".format(", ".join(s["name"] for s in SYSTEMS)))
    print("Output: {}".format(OUTPUT_FILE))
    print("Started: {}".format(datetime.now().isoformat()))
    print()

    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    all_rows = []

    with TCPython() as session:
        print("Connected to Thermo-Calc\n")

        for sys_def in SYSTEMS:
            sys_name = sys_def["name"]
            metal = sys_def["metal"]
            oxide = sys_def["oxide"]
            metal_frac = sys_def["metal_frac"]
            O_frac = sys_def["O_frac"]

            print("=" * 60)
            print("System: {} (oxide = {})".format(sys_name, oxide))
            print("=" * 60)

            try:
                system = (session
                          .select_database_and_elements(DATABASE, sys_def["elements"])
                          .get_system())
            except Exception as e:
                print("  SYSTEM SETUP ERROR: {}".format(e))
                for X_Cu in X_CU_VALUES:
                    all_rows.append({
                        "system": sys_name,
```

```
                        "oxide": oxide,
                        "T_K": T_FIXED,
                        "X_Cu": X_Cu,
                        "X_M": "",
                        "X_O": "",
                        "a_Cu": "",
                        "stable_phases": "ERROR: {}".format(e),
                        "notes": "System setup failed",
                    })
                continue

            success = 0
            for X_Cu in X_CU_VALUES:
                row = {
                    "system": sys_name,
                    "oxide": oxide,
                    "T_K": T_FIXED,
                    "X_Cu": round(X_Cu, 6),
                }

                # Non-Cu fraction distributed in oxide stoichiometry
                X_oxide_total = 1.0 - X_Cu
                X_M = X_oxide_total * metal_frac
                X_O = X_oxide_total * O_frac
                row["X_M"] = round(X_M, 6)
                row["X_O"] = round(X_O, 6)

                try:
                    calc = system.with_single_equilibrium_calculation()
                    calc.set_condition(ThermodynamicQuantity.temperature(), T_FIXED)
                    calc.set_condition(ThermodynamicQuantity.pressure(), 101325)
                    calc.set_condition(
                        ThermodynamicQuantity.mole_fraction_of_a_component("CU"),
                        X_Cu
                    )
                    calc.set_condition(
                        ThermodynamicQuantity.mole_fraction_of_a_component("O"),
                        X_O
                    )
                    result = calc.calculate()

                    a_Cu = result.get_value_of("AC(CU)")
                    stable = result.get_stable_phases()

                    row["a_Cu"] = a_Cu
                    row["stable_phases"] = "; ".join(stable)
                    row["notes"] = ""
                    success += 1

                except Exception as e:
                    row["a_Cu"] = ""
                    row["stable_phases"] = "ERROR: {}".format(e)
                    row["notes"] = str(e)

                all_rows.append(row)

            print("  Completed: {}/{} compositions".format(success, N_STEPS))

            # Print a few key values
            sys_rows = [r for r in all_rows
                        if r["system"] == sys_name and r.get("a_Cu", "") != ""]
            if sys_rows:
                first = sys_rows[0]
                last = sys_rows[-1]
                print("  a_Cu at X_Cu={:.2f}: {:.4f}".format(
                    first["X_Cu"], first["a_Cu"]))
                print("  a_Cu at X_Cu={:.2f}: {:.4f}".format(
                    last["X_Cu"], last["a_Cu"]))
            print()

```

```
    # ======================================================================
    # Write CSV
    # ======================================================================
    fieldnames = [
        "system", "oxide", "T_K", "X_Cu", "X_M", "X_O",
        "a_Cu", "stable_phases", "notes",
    ]

    with open(OUTPUT_FILE, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(all_rows)

    print("=" * 70)
    print("CSV written to: {}".format(OUTPUT_FILE))
    print("{} rows ({} systems x {} compositions)".format(
        len(all_rows), len(SYSTEMS), N_STEPS))
    print("Finished: {}".format(datetime.now().isoformat()))
    print("=" * 70)

    # Summary table
    print("\n" + "=" * 70)
    print("SUMMARY: Cu Activity at 1800K")
    print("=" * 70)
    print("{:<12} {:>10} {:>10} {:>10}".format(
        "System", "a_Cu(0.90)", "a_Cu(0.50)", "a_Cu(0.10)"))
    print("-" * 46)

    for sys_def in SYSTEMS:
        sys_name = sys_def["name"]
        sys_rows = [r for r in all_rows
                    if r["system"] == sys_name and r.get("a_Cu", "") != ""]
        vals = {}
        for r in sys_rows:
            for target in [0.90, 0.50, 0.10]:
                if abs(r["X_Cu"] - target) < 0.03:
                    vals[target] = r["a_Cu"]
        line = "{:<12}".format(sys_name)
        for target in [0.90, 0.50, 0.10]:
            if target in vals:
                line += " {:>10.4f}".format(vals[target])
            else:
                line += " {:>10}".format("N/A")
        print(line)

    print("=" * 70)


if __name__ == "__main__":
    main()
```

D.5 slag_composition_effects.py

Runs on: OSU ETS VM (requires TC-Python + TCOX14 [4])
Runtime: ∼5 minutes
Output: `data/tcpython/raw/slag_composition_effects.csv`
Used in: Section 6.4 (Slag Basicity Results), Fig. 5

Tests whether slag basicity affects Cu capture. Fixes $x_{\mathrm{Cu}} = 0.003$ (steelmaking level) and varies the basic-to-acidic oxide ratio in two quaternary systems: Cu-Mn-Si-O and Cu-Al-Si-O. If a_{Cu} decreases with increasing basicity, a more basic slag promotes Cu capture. Result: MnO:SiO_2 ratio is a tunable knob (-17% a_{Cu}); Al_2O_3:SiO_2 has no effect.

```
#!/usr/bin/env python3
"""
Test whether slag basicity affects Cu capture.

Quaternary systems: Cu-Mn-Si-O, Cu-Al-Si-O
Fix X_Cu = 0.003 (steelmaking Cu level ~0.3 wt%), vary the ratio
of basic oxide (MnO or Al2O3) to acidic oxide (SiO2).

For Cu-Mn-Si-O:
  Vary MnO:SiO2 ratio from 0.2 to 3.0 (basic to very basic slag)
  At each ratio, set X_Mn, X_Si, X_O to maintain stoichiometry.

For Cu-Al-Si-O:
  Vary Al2O3:SiO2 ratio from 0.2 to 3.0
  At each ratio, set X_Al, X_Si, X_O to maintain stoichiometry.

Record a_Cu and stable phases at each basicity.
If a_Cu decreases with basicity, basic slag promotes Cu capture.

T = 1800K (steelmaking), P = 101325 Pa.

Output: ../../data/tcpython/raw/slag_composition_effects.csv

Run on OSU lab machine:
  "C:\\Program Files\\Thermo-Calc\\2025b\\python\\python.exe" slag_composition_effects.py
"""

import csv
from pathlib import Path
from datetime import datetime

from tc_python import *

# ============================================================================
# Configuration
# ============================================================================
T_FIXED = 1800  # K
DATABASE = "TCOX14"
X_CU_FIXED = 0.003  # ~0.3 wt% Cu in steel

N_STEPS = 15
# Basicity ratio (basic_oxide : SiO2) from 0.2 to 3.0
RATIO_MIN = 0.2
RATIO_MAX = 3.0
RATIOS = [RATIO_MIN + i * (RATIO_MAX - RATIO_MIN) / (N_STEPS - 1)
          for i in range(N_STEPS)]

SCRIPT_DIR = Path(__file__).parent
OUTPUT_DIR = SCRIPT_DIR.parent.parent / "data" / "tcpython" / "raw"
OUTPUT_FILE = OUTPUT_DIR / "slag_composition_effects.csv"

# ============================================================================
# System definitions
#
# For Cu-Mn-Si-O with ratio R = MnO:SiO2 (in formula units):
#   R formula units MnO  = R Mn + R O atoms
#   1 formula unit SiO2 = 1 Si + 2 O atoms
```

```
#   Total non-Cu atoms = R + R + 1 + 2 = 2R + 3
#   X_Mn = R / (2R + 3), X_Si = 1 / (2R + 3), X_O = (R + 2) / (2R + 3)
#   Then scale by (1 - X_Cu) and add X_Cu.
#
# For Cu-Al-Si-O with ratio R = Al2O3:SiO2 (in formula units):
#   R formula units Al2O3 = 2R Al + 3R O atoms
#   1 formula unit SiO2 = 1 Si + 2 O atoms
#   Total non-Cu atoms = 2R + 3R + 1 + 2 = 5R + 3
#   X_Al = 2R / (5R + 3), X_Si = 1 / (5R + 3), X_O = (3R + 2) / (5R + 3)
# ============================================================================

SYSTEMS = [
    {
        "name": "Cu-Mn-Si-O",
        "elements": ["CU", "MN", "SI", "O"],
        "basic_oxide": "MnO",
        "ratio_label": "MnO:SiO2",
        # Given ratio R, compute non-Cu mole fractions
        # (before scaling by 1 - X_Cu)
        "calc_fracs": lambda R: {
            "MN": R / (2*R + 3),
            "SI": 1 / (2*R + 3),
            "O":  (R + 2) / (2*R + 3),
        },
        # For TC: set 3 conditions (CU, MN, O), SI fills remainder
        "set_conditions": ["CU", "MN", "O"],
    },
    {
        "name": "Cu-Al-Si-O",
        "elements": ["CU", "AL", "SI", "O"],
        "basic_oxide": "Al2O3",
        "ratio_label": "Al2O3:SiO2",
        "calc_fracs": lambda R: {
            "AL": 2*R / (5*R + 3),
            "SI": 1 / (5*R + 3),
            "O":  (3*R + 2) / (5*R + 3),
        },
        # For TC: set 3 conditions (CU, AL, O), SI fills remainder
        "set_conditions": ["CU", "AL", "O"],
    },
]


def main():
    print("=" * 70)
    print("TC-Python: Slag Composition Effects on Cu Activity at {}K".format(T_FIXED))
    print("=" * 70)
    print("Database: {}".format(DATABASE))
    print("X_Cu fixed: {}".format(X_CU_FIXED))
    print("Basicity ratios: {:.1f} to {:.1f} ({} steps)".format(
        RATIO_MIN, RATIO_MAX, N_STEPS))
    print("Systems: {}".format(", ".join(s["name"] for s in SYSTEMS)))
    print("Output: {}".format(OUTPUT_FILE))
    print("Started: {}".format(datetime.now().isoformat()))
    print()

    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    all_rows = []

    with TCPython() as session:
        print("Connected to Thermo-Calc\n")

        for sys_def in SYSTEMS:
            sys_name = sys_def["name"]
            ratio_label = sys_def["ratio_label"]

            print("=" * 60)
            print("System: {} (ratio = {})".format(sys_name, ratio_label))
            print("=" * 60)
```

```python

            try:
                system = (session
                          .select_database_and_elements(DATABASE, sys_def["elements"])
                          .get_system())
            except Exception as e:
                print("  SYSTEM SETUP ERROR: {}".format(e))
                for R in RATIOS:
                    all_rows.append({
                        "system": sys_name,
                        "ratio_label": ratio_label,
                        "ratio_value": round(R, 4),
                        "T_K": T_FIXED,
                        "X_Cu": X_CU_FIXED,
                        "X_M1": "", "X_Si": "", "X_O": "",
                        "a_Cu": "",
                        "stable_phases": "ERROR: {}".format(e),
                        "notes": "System setup failed",
                    })
                continue

            success = 0
            for R in RATIOS:
                # Compute slag oxide fractions (non-Cu portion)
                slag_fracs = sys_def["calc_fracs"](R)

                # Scale to total composition
                scale = 1.0 - X_CU_FIXED
                X_Cu = X_CU_FIXED

                # Element fractions after scaling
                comp = {}
                for el, frac in slag_fracs.items():
                    comp[el] = frac * scale

                # Get the metal (non-Si, non-O) fraction for display
                metal_els = [el for el in slag_fracs if el not in ("SI", "O")]
                X_M1 = comp.get(metal_els[0], 0) if metal_els else 0
                X_Si = comp.get("SI", 0)
                X_O = comp.get("O", 0)

                row = {
                    "system": sys_name,
                    "ratio_label": ratio_label,
                    "ratio_value": round(R, 4),
                    "T_K": T_FIXED,
                    "X_Cu": X_CU_FIXED,
                    "X_M1": round(X_M1, 6),
                    "X_Si": round(X_Si, 6),
                    "X_O": round(X_O, 6),
                }

                try:
                    calc = system.with_single_equilibrium_calculation()
                    calc.set_condition(ThermodynamicQuantity.temperature(), T_FIXED)
                    calc.set_condition(ThermodynamicQuantity.pressure(), 101325)

                    # Set 3 mole fraction conditions; 4th element (SI) fills remainder
                    cond_els = sys_def["set_conditions"]
                    for el in cond_els:
                        val = X_Cu if el == "CU" else comp[el]
                        calc.set_condition(
                            ThermodynamicQuantity.mole_fraction_of_a_component(el),
                            val
                        )

                    result = calc.calculate()

                    a_Cu = result.get_value_of("AC(CU)")
                    stable = result.get_stable_phases()
```

```python

                    row["a_Cu"] = a_Cu
                    row["stable_phases"] = "; ".join(stable)
                    row["notes"] = ""
                    success += 1

                except Exception as e:
                    row["a_Cu"] = ""
                    row["stable_phases"] = "ERROR: {}".format(e)
                    row["notes"] = str(e)

                all_rows.append(row)

            print("  Completed: {}/{} ratios".format(success, N_STEPS))

            # Print key values
            sys_rows = [r for r in all_rows
                        if r["system"] == sys_name and r.get("a_Cu", "") != ""]
            if sys_rows:
                first = sys_rows[0]
                last = sys_rows[-1]
                print("  a_Cu at R={:.1f}: {:.6f}".format(
                    first["ratio_value"], first["a_Cu"]))
                print("  a_Cu at R={:.1f}: {:.6f}".format(
                    last["ratio_value"], last["a_Cu"]))

                # Check monotonicity
                a_vals = [r["a_Cu"] for r in sys_rows]
                diffs = [a_vals[i+1] - a_vals[i] for i in range(len(a_vals)-1)]
                if all(d <= 0 for d in diffs):
                    print("  Trend: monotonically decreasing (more basic = lower a_Cu)")
                elif all(d >= 0 for d in diffs):
                    print("  Trend: monotonically increasing (more basic = higher a_Cu)")
                else:
                    print("  Trend: non-monotonic")
            print()

    # ======================================================================
    # Write CSV
    # ======================================================================
    fieldnames = [
        "system", "ratio_label", "ratio_value", "T_K",
        "X_Cu", "X_M1", "X_Si", "X_O",
        "a_Cu", "stable_phases", "notes",
    ]

    with open(OUTPUT_FILE, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(all_rows)

    print("=" * 70)
    print("CSV written to: {}".format(OUTPUT_FILE))
    print("{} rows ({} systems x {} ratios)".format(
        len(all_rows), len(SYSTEMS), N_STEPS))
    print("Finished: {}".format(datetime.now().isoformat()))
    print("=" * 70)


if __name__ == "__main__":
    main()
```

D.6 compute_ternary_dG.py

Runs on: Local (post-processing, no TC-Python needed)
Runtime: <1 second
Input: `data/tcpython/raw/ternary_reaction_energies.csv`
Output: `screening/ternary_screening_results.csv`
Used in: Section 4, Ternary_Verdicts tab of unified workbook

Reads the raw ternary reaction energies from Script 2 and produces the ranked summary table: product name, parent oxide, ΔG at three key temperatures (727, 1227, 1527 °C), stable phases, and a FAVORABLE/UNFAVORABLE verdict. This is the script that confirmed all 16 reactions are favorable.

```python
"""
Post-process ternary reaction energies from TC-Python extraction.

Reads ternary_reaction_energies.csv and produces:
  1. Summary table of all ternary reactions at steelmaking temperatures
  2. Ranked list of most promising oxide candidates
  3. CSV output for downstream visualization

Run locally after copying ternary_reaction_energies.csv from OSU VM.
"""

import csv
import numpy as np
from pathlib import Path

SCRIPT_DIR = Path(__file__).parent
CSV_IN = SCRIPT_DIR.parent / "data" / "tcpython" / "raw" / "ternary_reaction_energies.csv"
CSV_OUT = SCRIPT_DIR / "ternary_screening_results.csv"

# Key steelmaking temperatures
TEMPS_K = [1000, 1500, 1800]  # 727, 1227, 1527 C


def main():
    if not CSV_IN.exists():
        print(f"ERROR: {CSV_IN} not found!")
        print("Run extract_ternary_reactions.py on OSU VM first, then copy the CSV here.")
        return

    with open(CSV_IN) as f:
        reader = csv.DictReader(f)
        rows = list(reader)

    print("=" * 80)
    print("Ternary Reaction Screening -- Cu + MOx → CuMOy")
    print("Negative dG_rxn = FAVORABLE (copper captured into ternary compound)")
    print("=" * 80)

    # Get unique products
    products = []
    seen = set()
    for r in rows:
        key = r["product"]
        if key not in seen:
            seen.add(key)
            products.append({
                "product": r["product"],
                "product_name": r["product_name"],
                "oxide": r["oxide"],
                "reaction": r["reaction"],
            })

    # Extract dG at key temperatures for each product
    results = []
    for prod_info in products:
        product = prod_info["product"]
        prod_rows = [r for r in rows if r["product"] == product]
```

```python

        # Build temperature -> dG map
        T_vals = []
        dG_vals = []
        for r in prod_rows:
            T = float(r["T_K"])
            dG_str = r.get("dG_rxn_system_kJ", "").strip()
            if dG_str:
                try:
                    dG = float(dG_str)
                    T_vals.append(T)
                    dG_vals.append(dG)
                except ValueError:
                    pass

        T_arr = np.array(T_vals)
        dG_arr = np.array(dG_vals)

        entry = {
            "Product": product,
            "Name": prod_info["product_name"],
            "Oxide": prod_info["oxide"],
            "Reaction": prod_info["reaction"],
        }

        # Get dG at key temperatures
        for T_target in TEMPS_K:
            key = f"dG_rxn @ {T_target - 273}°C (kJ)"
            if len(T_arr) > 0:
                idx = int(np.argmin(np.abs(T_arr - T_target)))
                if np.abs(T_arr[idx] - T_target) < 30:  # within 30K
                    entry[key] = f"{dG_arr[idx]:+.1f}"
                else:
                    entry[key] = "No data near this T"
            else:
                entry[key] = "No data"

        # Check for ternary phase at 1800K
        rows_1800 = [r for r in prod_rows
                     if abs(float(r["T_K"]) - 1800) < 30]
        if rows_1800:
            entry["Ternary Phase Found"] = rows_1800[0].get("ternary_phase_found", "")
            entry["Stable Phases @ 1527°C"] = rows_1800[0].get("stable_phases", "")
        else:
            entry["Ternary Phase Found"] = ""
            entry["Stable Phases @ 1527°C"] = ""

        # Verdict: use 1800K value
        dG_1800_key = f"dG_rxn @ {1800 - 273}°C (kJ)"
        dG_str = entry.get(dG_1800_key, "")
        if dG_str and dG_str not in ("No data", "No data near this T"):
            dG_val = float(dG_str)
            if dG_val < -10:
                entry["Verdict"] = "FAVORABLE -- Cu captured"
            elif dG_val < 0:
                entry["Verdict"] = "Marginally favorable"
            elif dG_val < 10:
                entry["Verdict"] = "Borderline"
            else:
                entry["Verdict"] = "Unfavorable"
        else:
            entry["Verdict"] = "No data"

        results.append(entry)

    # Print table
    print(f"\n{'Product':<16} ", end="")
    for T in TEMPS_K:
        print(f"  {'@' + str(T-273) + '°C':>14}", end="")
    print(f"  {'Ternary Phase':>16}  {'Verdict':<24}")
```

```python
    print("-" * 110)

    for r in results:
        print(f"{r['Product']:<16} ", end="")
        for T in TEMPS_K:
            key = f"dG_rxn @ {T - 273}°C (kJ)"
            print(f"  {r[key]:>14}", end="")
        phase = r.get("Ternary Phase Found", "")
        print(f"  {(phase or '--'):>16}", end="")
        print(f"  {r['Verdict']:<24}")

    # Rank by dG at 1527°C
    print(f"\n{'='*80}")
    print("RANKING: Most Promising Oxides for Copper Capture (1527°C)")
    print("=" * 80)

    dG_key = f"dG_rxn @ {1800 - 273}°C (kJ)"
    ranked = []
    for r in results:
        v = r.get(dG_key, "")
        if v and v not in ("No data", "No data near this T"):
            ranked.append((r["Product"], float(v), r["Oxide"], r.get("Ternary Phase Found", "")))

    ranked.sort(key=lambda x: x[1])

    for i, (product, dG, oxide, phase) in enumerate(ranked, 1):
        marker = "***" if dG < 0 else "   "
        print(f"  {marker} {i:2d}. {product:<16}  dG = {dG:>+8.1f} kJ   (from {oxide})"
              f"  phase: {phase or 'not found'}")

    # Write CSV
    if results:
        fieldnames = list(results[0].keys())
        with open(CSV_OUT, 'w', newline='') as f:
            writer = csv.DictWriter(f, fieldnames=fieldnames)
            writer.writeheader()
            writer.writerows(results)
        print(f"\nCSV written to: {CSV_OUT}")

    # Identify top candidates
    favorable = [r for r in ranked if r[1] < 0]
    if favorable:
        print(f"\n*** {len(favorable)} FAVORABLE reactions found! ***")
        print("These oxides can capture copper into ternary compounds:")
        for product, dG, oxide, phase in favorable:
            print(f"   {oxide} → {product}  (dG = {dG:+.1f} kJ, phase: {phase or 'check manually'})")
        print("\nRecommend these for experimental validation.")
    else:
        print("\nNo strongly favorable ternary reactions found at 1527°C.")
        print("Check lower temperatures or consider kinetic/slag mechanisms.")


if __name__ == "__main__":
    main()
```

D.7 compute_activity_corrected_dG.py

Runs on: Local (post-processing, no TC-Python needed)
Runtime: ∼2 seconds
Input: `data/tcpython/raw/ternary_reaction_energies.csv`
Output: `data/tcpython/processed/activity_corrected_dG.csv`, sensitivity figures
Used in: Section 5 (Activity Correction), Figs. 3, 4

The script that reshuffled everything. Takes the pure-Cu ΔG values and applies the dilute-Cu activity penalty: $\Delta G_{\text{eff}} = \Delta G_{\text{pure}} + n_{\text{Cu}} \cdot RT \ln(a_{\text{Cu}})$. Uses $\gamma_{\text{Cu}} = 8.5$ [9] and $x_{\text{Cu}} = 0.003$. Also runs a sensitivity sweep over $\gamma_{\text{Cu}} = 3$–15 to test sensitivity. This is where $Cu_3V_2O_8$ (ranked #2 by pure ΔG) gets eliminated and $CuFe_2O_4$ emerges as the clear winner.

```python
"""
Compute activity-corrected dG for ternary reactions under steelmaking conditions.

Our existing dG values assume PURE Cu and PURE O2 as reactants:
    dG_pure = G(CuMOy) - G(Cu_pure) - n*G(O2_pure) - G(MOx_pure)

In real steel, Cu is dilute (X_Cu ~ 0.003, a_Cu << 1) and O2 activity is set
by the deoxidation practice. The effective driving force is:
    dG_eff = dG_pure + RT * ln(a_Cu) + n * RT * ln(pO2/1atm)

Since ln(a_Cu) is negative (a_Cu < 1), the RT*ln(a_Cu) term is POSITIVE,
making reactions LESS favorable. This correction preserves the ranking but
changes which reactions remain thermodynamically viable.

IMPORTANT: For each parent oxide, we evaluate ALL possible products and pick
the one with the most favorable corrected dG. Under dilute Cu conditions,
products consuming fewer Cu atoms are preferred (e.g., CuV2O6 over Cu3V2O8).

Output: data/tcpython/processed/activity_corrected_dG.csv
        figures/dG_corrected_comparison.png + .pdf
        figures/dG_sensitivity_gamma_Cu.png + .pdf
"""

import csv
import math
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from pathlib import Path

SCRIPT_DIR = Path(__file__).parent

# Primary data source: fine-resolution top 6 (25K steps)
CSV_TOP6 = SCRIPT_DIR.parent / "data" / "tcpython" / "raw" / "dG_vs_T_top6.csv"
# Full data source: all products (50K steps) -- for products not in top 6
CSV_FULL = SCRIPT_DIR.parent / "data" / "tcpython" / "raw" / "ternary_reaction_energies.csv"

CSV_OUT = SCRIPT_DIR.parent / "data" / "tcpython" / "processed" / "activity_corrected_dG.csv"
FIG_DIR = SCRIPT_DIR.parent / "figures"

R = 8.314  # J/(mol*K)

# Steelmaking Cu level
X_CU = 0.003  # ~0.3 wt% Cu in steel

# For dilute solution in liquid iron, a_Cu = gamma * X_Cu
# gamma_Cu in liquid Fe at 1800K: literature range 5-13 (Raoultian)
# Hino & Ito (2010) recommend ~8.5; Sigworth & Elliott (1974) give ~10
GAMMA_CU = 8.5
A_CU = GAMMA_CU * X_CU  # ~0.0255

# Products and their O2 stoichiometric coefficients in the reaction
# Cu + MOx + n*O2 -> CuMOy
O2_COEFFICIENTS = {
```

```
    "CuFe2O4": 1.0,      # Cu + 2FeO + O2 -> CuFe2O4
    "Cu3V2O8": 1.5,      # 3Cu + V2O5 + 1.5O2 -> Cu3V2O8
    "CuV2O6":  0.5,      # Cu + V2O5 + 0.5O2 -> CuV2O6
    "CuMn2O4": 1.0,      # Cu + 2MnO + O2 -> CuMn2O4
    "Cu2SiO4": 1.0,      # 2Cu + SiO2 + O2 -> Cu2SiO4
    "CuB2O4":  0.5,      # Cu + B2O3 + 0.5O2 -> CuB2O4
    "CuAl2O4": 0.5,      # Cu + Al2O3 + 0.5O2 -> CuAl2O4
}

# Number of Cu atoms consumed per formula unit
CU_ATOMS = {
    "CuFe2O4": 1,
    "Cu3V2O8": 3,
    "CuV2O6":  1,
    "CuMn2O4": 1,
    "Cu2SiO4": 2,
    "CuB2O4":  1,
    "CuAl2O4": 1,
}

# Plot order: by corrected dG (most favorable first)
PRODUCT_ORDER = [
    "CuFe2O4", "CuMn2O4", "CuV2O6", "CuB2O4",
    "CuAl2O4", "Cu3V2O8", "Cu2SiO4",
]

COLORS = {
    "CuFe2O4": "#0077BB",
    "Cu3V2O8": "#EE7733",
    "CuV2O6":  "#CC6600",
    "CuMn2O4": "#AA3377",
    "Cu2SiO4": "#009988",
    "CuB2O4":  "#EE3377",
    "CuAl2O4": "#BBBBBB",
}

LINE_STYLES = {
    "CuFe2O4": "-",
    "Cu3V2O8": "--",
    "CuV2O6":  "-.",
    "CuMn2O4": ":",
    "Cu2SiO4": "-.",
    "CuB2O4":  "-",
    "CuAl2O4": "--",
}

SHORT_LABELS = {
    "CuFe2O4": r"CuFe$_2$O$_4$",
    "Cu3V2O8": r"Cu$_3$V$_2$O$_8$",
    "CuV2O6":  r"CuV$_2$O$_6$",
    "CuMn2O4": r"CuMn$_2$O$_4$",
    "Cu2SiO4": r"Cu$_2$SiO$_4$",
    "CuB2O4":  r"CuB$_2$O$_4$",
    "CuAl2O4": r"CuAl$_2$O$_4$",
}

# Parent oxide for each product (for sensitivity analysis grouping)
PARENT_OXIDE = {
    "CuFe2O4": "FeO",
    "Cu3V2O8": "V2O5",
    "CuV2O6":  "V2O5",
    "CuMn2O4": "MnO",
    "Cu2SiO4": "SiO2",
    "CuB2O4":  "B2O3",
    "CuAl2O4": "Al2O3",
}


def load_data():
    """Load dG data from both CSV sources."""
```

```
    data = {}

    # First load fine-resolution top-6 data
    if CSV_TOP6.exists():
        with open(CSV_TOP6) as f:
            for r in csv.DictReader(f):
                product = r["product"]
                if product not in O2_COEFFICIENTS:
                    continue
                if product not in data:
                    data[product] = []
                try:
                    T = float(r["T_K"])
                    dG_pure = float(r["dG_rxn_system_kJ"])
                    data[product].append((T, dG_pure))
                except (ValueError, KeyError):
                    pass

    # Then load any missing products from the full ternary CSV
    # Only load products we don't already have from the fine-resolution CSV
    products_from_top6 = set(data.keys())
    if CSV_FULL.exists():
        with open(CSV_FULL) as f:
            for r in csv.DictReader(f):
                product = r.get("product", "")
                if product not in O2_COEFFICIENTS:
                    continue
                if product in products_from_top6:
                    continue  # already have fine-resolution data
                if product not in data:
                    data[product] = []
                try:
                    T = float(r["T_K"])
                    dG_kJ_str = r.get("dG_rxn_system_kJ", "")
                    if not dG_kJ_str:
                        continue
                    dG_pure = float(dG_kJ_str)
                    data[product].append((T, dG_pure))
                except (ValueError, KeyError):
                    pass

    return data


def main():
    print("=" * 70)
    print("Activity-Corrected dG for Ternary Reactions")
    print("=" * 70)
    print("X_Cu = %.4f, gamma_Cu = %.1f, a_Cu = %.4f" % (X_CU, GAMMA_CU, A_CU))
    print("RT*ln(a_Cu) at 1800K = %.1f kJ/mol" % (R * 1800 * math.log(A_CU) / 1000))
    print()

    data = load_data()

    # Report what we loaded
    for product in PRODUCT_ORDER:
        n = len(data.get(product, []))
        src = "top6" if product in ["CuFe2O4", "Cu3V2O8", "CuMn2O4",
                                    "Cu2SiO4", "CuB2O4", "CuAl2O4"] and n > 30 \
              else "ternary_full"
        print("  %-14s: %3d data points (source: %s)" % (product, n, src))
    print()

    # Compute corrections
    CSV_OUT.parent.mkdir(parents=True, exist_ok=True)
    FIG_DIR.mkdir(parents=True, exist_ok=True)

    out_rows = []

    print("%-14s  %4s  %8s  %8s  %8s  %8s  %s" % (
```

```
        "Product", "nCu", "dG_pure", "Cu_corr", "dG_corr", "dG_air", "Verdict"))
    print("-" * 75)

    for product in PRODUCT_ORDER:
        if product not in data:
            continue

        n_cu = CU_ATOMS[product]
        n_o2 = O2_COEFFICIENTS[product]

        for T, dG_pure in sorted(data[product]):
            # Cu activity correction: Cu is a REACTANT, so lowering its
            # activity raises dG (less favorable). For reaction quotient Q:
            #   dG = dG_pure + RT*ln(Q)
            #   Q = a_products / (a_Cu^n_Cu * a_O2^n_O2 * a_MOx)
            # With a_products=1, a_MOx=1 (pure solid oxide):
            #   RT*ln(Q) = -n_Cu*RT*ln(a_Cu) - n_O2*RT*ln(pO2)
            cu_correction_kJ = -n_cu * R * T * math.log(A_CU) / 1000.0

            # O2 correction at 1 atm: zero (ln(1) = 0)
            # At 0.21 atm: -n_O2 * RT * ln(0.21) > 0 (less favorable)
            o2_correction_air_kJ = -n_o2 * R * T * math.log(0.21) / 1000.0

            # Case A: pO2 = 1 atm, dilute Cu
            dG_corrected_A = dG_pure + cu_correction_kJ

            # Case B: pO2 = 0.21 atm (air), dilute Cu
            dG_corrected_B = dG_pure + cu_correction_kJ + o2_correction_air_kJ

            out_rows.append({
                "product": product,
                "T_K": T,
                "T_C": T - 273.15,
                "dG_pure_kJ": round(dG_pure, 2),
                "Cu_correction_kJ": round(cu_correction_kJ, 2),
                "O2_correction_air_kJ": round(o2_correction_air_kJ, 2),
                "dG_corrected_pO2_1atm_kJ": round(dG_corrected_A, 2),
                "dG_corrected_pO2_air_kJ": round(dG_corrected_B, 2),
                "a_Cu": round(A_CU, 4),
                "n_Cu": n_cu,
                "n_O2": n_o2,
                "parent_oxide": PARENT_OXIDE[product],
                "verdict_corrected": "FAVORABLE" if dG_corrected_A < 0 else "UNFAVORABLE",
            })

            # Print 1800K values
            if abs(T - 1800) < 1:
                verdict = "FAVORABLE" if dG_corrected_A < 0 else "UNFAVORABLE"
                print("%-14s  %4d  %8.1f  %8.1f  %8.1f  %8.1f  %s" % (
                    product, n_cu, dG_pure, cu_correction_kJ,
                    dG_corrected_A, dG_corrected_B, verdict))

    # Write CSV
    fieldnames = [
        "product", "parent_oxide", "T_K", "T_C", "dG_pure_kJ", "Cu_correction_kJ",
        "O2_correction_air_kJ", "dG_corrected_pO2_1atm_kJ",
        "dG_corrected_pO2_air_kJ", "a_Cu", "n_Cu", "n_O2", "verdict_corrected",
    ]
    with open(CSV_OUT, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(out_rows)

    print()
    print("CSV saved: %s" % CSV_OUT)

    # ======================================================================
    # Plot 1: Pure vs Corrected comparison (both panels)
    # ======================================================================
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
```

```python

    for product in PRODUCT_ORDER:
        if product not in data:
            continue

        pts = sorted(data[product])
        T_arr = np.array([p[0] for p in pts])
        dG_arr = np.array([p[1] for p in pts])
        T_C = T_arr - 273.15

        n_cu = CU_ATOMS[product]
        # Correction: -n_Cu * RT * ln(a_Cu) is POSITIVE (makes dG less negative)
        cu_corr = -n_cu * R * T_arr * np.log(A_CU) / 1000.0
        dG_corrected = dG_arr + cu_corr

        # Left panel: pure reference
        ax1.plot(T_C, dG_arr,
                 color=COLORS[product],
                 linestyle=LINE_STYLES[product],
                 linewidth=1.8,
                 label=SHORT_LABELS[product])

        # Right panel: activity-corrected
        ax2.plot(T_C, dG_corrected,
                 color=COLORS[product],
                 linestyle=LINE_STYLES[product],
                 linewidth=1.8,
                 label=SHORT_LABELS[product])

    for ax in [ax1, ax2]:
        ax.axhline(y=0, color='black', linewidth=0.8, linestyle='-', alpha=0.5)
        ax.axvspan(1500, 1650, color='#CCCCCC', alpha=0.25, zorder=0)
        ax.set_xlabel(r"Temperature ($^\circ$C)", fontsize=12)
        ax.set_ylabel(r"$\Delta G_{\mathrm{rxn}}$ (kJ/mol)", fontsize=12)
        ax.grid(True, which='major', alpha=0.3)
        ax.minorticks_on()
        ax.grid(True, which='minor', alpha=0.1)
        for spine in ax.spines.values():
            spine.set_visible(False)

    # Set y-limits after plotting so text placement works
    for ax in [ax1, ax2]:
        ymin, ymax = ax.get_ylim()
        ax.text(1575, ymax - 0.05 * (ymax - ymin), "Steelmaking\nrange",
                fontsize=8, color='#777777', ha='center', va='top', style='italic')

    ax1.set_title("Pure Reference States\n" + r"($a_{\mathrm{Cu}}$ = 1)", fontsize=11)
    ax2.set_title("Steelmaking Conditions\n" +
                  r"($a_{\mathrm{Cu}}$ = %.4f, X$_{\mathrm{Cu}}$ = %.3f)" % (A_CU, X_CU),
                  fontsize=11)

    ax1.legend(fontsize=7, loc='lower right', framealpha=0.9, edgecolor='#CCCCCC')
    ax2.legend(fontsize=7, loc='lower left', framealpha=0.9, edgecolor='#CCCCCC')

    plt.tight_layout()

    png_path = FIG_DIR / "dG_corrected_comparison.png"
    pdf_path = FIG_DIR / "dG_corrected_comparison.pdf"
    fig.savefig(png_path, dpi=300, bbox_inches='tight')
    fig.savefig(pdf_path, bbox_inches='tight')
    plt.close(fig)

    print("Figure saved: %s" % png_path)
    print("Figure saved: %s" % pdf_path)

    # ======================================================================
    # Plot 2: gamma_Cu sensitivity analysis at 1800K
    # ======================================================================
    gamma_range = np.linspace(3.0, 15.0, 50)
    T_sens = 1800.0
```

```python

    # Get dG_pure at 1800K for each product
    dG_pure_1800 = {}
    for product in PRODUCT_ORDER:
        if product not in data:
            continue
        for T, dG in data[product]:
            if abs(T - T_sens) < 1:
                dG_pure_1800[product] = dG
                break

    fig2, ax3 = plt.subplots(1, 1, figsize=(8, 5.5))

    for product in PRODUCT_ORDER:
        if product not in dG_pure_1800:
            continue

        dG_pure = dG_pure_1800[product]
        n_cu = CU_ATOMS[product]

        dG_eff = np.array([
            dG_pure + (-n_cu * R * T_sens * math.log(g * X_CU) / 1000.0)
            for g in gamma_range
        ])

        ax3.plot(gamma_range, dG_eff,
                 color=COLORS[product],
                 linestyle=LINE_STYLES[product],
                 linewidth=1.8,
                 label="%s (n$_{Cu}$=%d)" % (SHORT_LABELS[product], n_cu))

    ax3.axhline(y=0, color='black', linewidth=0.8, linestyle='-', alpha=0.5)
    ax3.axvspan(7.0, 10.0, color='#CCCCCC', alpha=0.2, zorder=0,
                label=r"Literature $\gamma_{\mathrm{Cu}}$ range")
    ax3.axvline(x=GAMMA_CU, color='black', linewidth=0.5, linestyle='--', alpha=0.4)

    ax3.set_xlabel(r"$\gamma_{\mathrm{Cu}}$ (Raoultian activity coefficient)", fontsize=12)
    ax3.set_ylabel(r"$\Delta G_{\mathrm{eff}}$ at 1800 K (kJ/mol)", fontsize=12)
    ax3.set_title(r"Sensitivity to $\gamma_{\mathrm{Cu}}$ at 1800 K, $X_{\mathrm{Cu}}$ = 0.003",
                  fontsize=11)
    ax3.legend(fontsize=7, loc='upper left', framealpha=0.9, edgecolor='#CCCCCC')
    ax3.grid(True, which='major', alpha=0.3)
    ax3.minorticks_on()
    ax3.grid(True, which='minor', alpha=0.1)
    for spine in ax3.spines.values():
        spine.set_visible(False)

    plt.tight_layout()
    sens_png = FIG_DIR / "dG_sensitivity_gamma_Cu.png"
    sens_pdf = FIG_DIR / "dG_sensitivity_gamma_Cu.pdf"
    fig2.savefig(sens_png, dpi=300, bbox_inches='tight')
    fig2.savefig(sens_pdf, bbox_inches='tight')
    plt.close(fig2)

    print("Figure saved: %s" % sens_png)
    print("Figure saved: %s" % sens_pdf)

    # ============================================================================
    # Summary at 1800K
    # ============================================================================
    print()
    print("=" * 75)
    print("RANKING AT 1800K - Activity-Corrected (a_Cu = %.4f)" % A_CU)
    print("=" * 75)
    print()

    results_1800 = [(r["product"], r["n_Cu"], r["dG_pure_kJ"],
                     r["dG_corrected_pO2_1atm_kJ"])
                    for r in out_rows if abs(r["T_K"] - 1800) < 1]
    results_1800.sort(key=lambda x: x[3])
```

```python

    print("%-4s %-14s %4s %10s %12s %10s" % (
        "Rank", "Product", "nCu", "dG_pure", "dG_corrected", "Status"))
    print("-" * 60)
    for i, (product, n_cu, dG_pure, dG_corr) in enumerate(results_1800, 1):
        if dG_corr < -10:
            status = "ROBUST"
        elif dG_corr < 0:
            status = "MARGINAL"
        elif dG_corr < 15:
            status = "UNCERTAIN"
        else:
            status = "UNFAVORABLE"
        print("%-4d %-14s %4d %10.1f %12.1f %10s" % (
            i, product, n_cu, dG_pure, dG_corr, status))

    n_favorable = sum(1 for _, _, _, dG in results_1800 if dG < 0)
    n_uncertain = sum(1 for _, _, _, dG in results_1800 if 0 <= dG < 15)
    n_unfavorable = sum(1 for _, _, _, dG in results_1800 if dG >= 15)
    print()
    print("%d favorable, %d uncertain (within gamma_Cu error), %d unfavorable" % (
        n_favorable, n_uncertain, n_unfavorable))

    # V2O5 comparison: which product is preferred?
    print()
    print("=" * 75)
    print("V2O5 PRODUCTS: Cu3V2O8 vs CuV2O6")
    print("=" * 75)
    for product in ["Cu3V2O8", "CuV2O6"]:
        for r in out_rows:
            if r["product"] == product and abs(r["T_K"] - 1800) < 1:
                print("  %-14s: n_Cu=%d, dG_pure=%+.1f, correction=%+.1f, dG_eff=%+.1f kJ" % (
                    product, r["n_Cu"], r["dG_pure_kJ"],
                    r["Cu_correction_kJ"], r["dG_corrected_pO2_1atm_kJ"]))
    print("  Under dilute Cu, CuV2O6 (1 Cu) is preferred over Cu3V2O8 (3 Cu)")
    print("  At hypothetical higher-Cu conditions (~1 wt%% Cu, a_Cu~0.13):")
    for product in ["Cu3V2O8", "CuV2O6"]:
        if product in dG_pure_1800:
            n_cu = CU_ATOMS[product]
            a_cu_bom = 8.5 * 0.015  # ~1 wt% Cu
            corr = -n_cu * R * 1800 * math.log(a_cu_bom) / 1000.0
            dG_eff_bom = dG_pure_1800[product] + corr
            print("    %-14s: dG_eff = %+.1f kJ (a_Cu=%.3f) -> %s" % (
                product, dG_eff_bom, a_cu_bom,
                "FAVORABLE" if dG_eff_bom < 0 else "UNFAVORABLE"))

    print("=" * 75)


if __name__ == "__main__":
    main()
```

Last updated: March 14, 2026

REFERENCES

[1] K. E. Daehn, A. Cabrera Serrenho, and J. M. Allwood, "Finding the most efficient way to remove residual copper from steel scrap," *Metall. Mater. Trans. B*, vol. 50, pp. 1225–1240, 2019. doi: 10.1007/s11663-019-01537-9

[2] H. L. Lukas, S. G. Fries, and B. Sundman, *Computational Thermodynamics: The Calphad Method*. Cambridge University Press, 2007.

[3] J.-O. Andersson, T. Helander, L. Höglund, P. Shi, and B. Sundman, "Thermo-Calc & DICTRA, computational tools for materials science," *Calphad*, vol. 26, no. 2, pp. 273–312, 2002. doi: 10.1016/S0364-5916(02)00037-8

[4] Thermo-Calc Software, "TCS Metal Oxide Solutions Database (TCOX14)," https://thermocalc.com/products/databases/metal-oxide-solutions/, accessed Mar. 2026.

[5] Thermo-Calc Software, "SGTE Substances Database (SSUB3)," https://thermocalc.com/products/databases/, accessed Mar. 2026.

[6] D. R. Gaskell and D. E. Laughlin, *Introduction to the Thermodynamics of Materials*, 7th ed. CRC Press, 2024.

[7] M. Hillert, *Phase Equilibria, Phase Diagrams and Phase Transformations: Their Thermodynamic Basis*, 2nd ed. Cambridge University Press, 2007.

[8] D. A. Porter, K. E. Easterling, and M. Y. Sherif, *Phase Transformations in Metals and Alloys*, 4th ed. CRC Press, 2021.

[9] M. Hino and K. Ito, Eds., *Thermodynamic Data for Steelmaking*. Tohoku University Press, 2010.

[10] G. K. Sigworth and J. F. Elliott, "The thermodynamics of liquid dilute iron alloys," *Metal Science*, vol. 8, pp. 298–310, 1974. doi: 10.1179/msc.1974.8.1.298

[11] S. Seetharaman, Ed., *Fundamentals of Metallurgy*. Woodhead Publishing, 2005.

[12] 911 Metallurgist, "Removing copper from molten steel scrap," https://www.911metallurgist.com/blog/removing-copper-molten-steel-scrap, 2017. Summarizes Bureau of Mines oxide injection experiments.

[13] J. Brown, "CALPHAD introduction," MSE 3321 Lecture, The Ohio State University, 2026.

[14] J. Brown and Y. Ji, "Equilibrium calculations (TC1)," MSE 3321 Lab Handout, The Ohio State University, 2026.

[15] B. Sundman, B. Jansson, and J.-O. Andersson, "The Thermo-Calc databank system," *Calphad*, vol. 9, no. 2, pp. 153–190, 1985. doi: 10.1016/0364-5916(85)90021-5